

Adminisztrációs felület backend - Osztály szintű dokumentáció

April 30, 2026

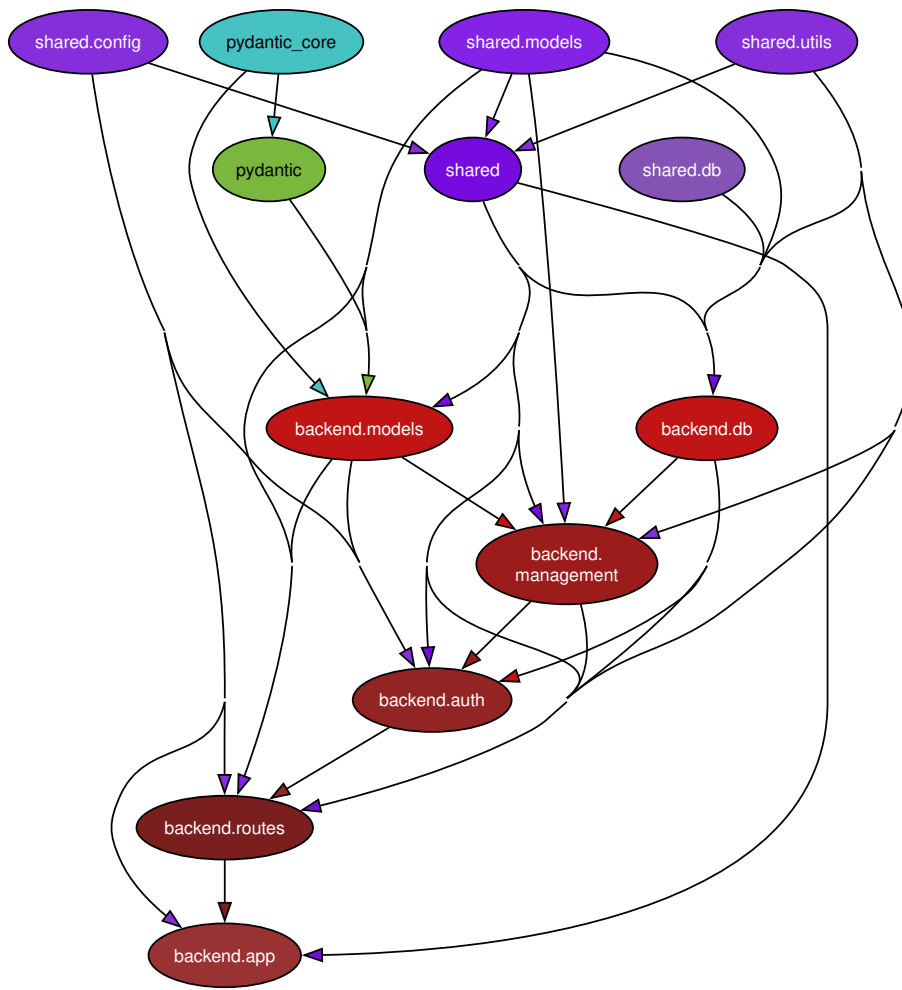


Figure 1: Az adminisztrációs felület backendjének `pydeps`-el létrehozott program csomag diagrammja.

1 backend.app package

1.0.1 `create_app()` → `fastapi.FastAPI`

Creates the FastAPI application instance, sets up the lifespan context, initializes middleware, and includes the API routers for health, authentication, user management, logs, keys, projects, and quotas.

1.1 Submodules

1.2 backend.app.factory module

1.2.1 create_app() → fastapi.FastAPI

Creates the FastAPI application instance, sets up the lifespan context, initializes middleware, and includes the API routers for health, authentication, user management, logs, keys, projects, and quotas.

1.2.2 lifespan(app: fastapi.FastAPI)

Defines the lifespan of the Fastapi application, handling startup and shutdown events.

1.2.3 async request_validation_exception_handler(_, exc: fastapi.exceptions.RequestValidationError)

1.3 backend.app.middleware module

1.3.1 init_middleware(app: fastapi.FastAPI)

Initializes the middleware for the FastAPI application, including the CORSMiddleware for handling Cross-Origin Resource Sharing (CORS) with permissive settings.

2 backend.auth package

2.0.1 create_access_token(data: AccessTokenInfo) → str

Create a JWT access token containing the user information and an expiration time.

- **Parameters:** data (*AccessTokenInfo*) – An *AccessTokenInfo* object containing the user ID and optionally the project ID to be included in the token payload.
- **Returns:** A JWT-encoded string that can be used as an access token for authenticating requests to protected endpoints.
- **Return type:** *str*

2.0.2 async get_user_from_token(token: str, project_id: int | None = None) → UserDisplayInformation | None

Extract the user information from a JWT access token. It decodes the token, verifies its validity, and retrieves the corresponding user record from the database.

- **Parameters:**

- **token** (*str*) – The JWT access token string to be decoded and validated.
- **project_id** (*int* / *None* , *optional*) – Optional project ID to include permissions in the information.
- **Returns:** A `UserDisplayInformation` object containing the user’s information if the token is valid and the user exists, or `None` if the token is invalid or the user cannot be found.
- **Return type:** `UserDisplayInformation` | `None`

2.0.3 `async require_administrator_user(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation`

A FastAPI dependency that ensures the current user is an administrator. It retrieves the user’s information using the provided JWT access token and checks if the user has administrator privileges. If the user is not an administrator, it raises an `HTTPException` with a 403 status code. This dependency is used for endpoints that require administrator-level access.

- **Parameters:** **credentials** (`HTTPAuthorizationCredentials` | `None` , *optional*) – An optional `HTTPAuthorizationCredentials` object provided by the `HTTPBearer` security scheme, containing the access token from the request headers.
- **Returns:** A `UserDisplayInformation` object containing the authenticated user’s information if the user is an administrator, or raises an `HTTPException` if the user is not an administrator or if authentication fails.
- **Return type:** `UserDisplayInformation`

2.0.4 `async require_current_user(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation`

A FastAPI dependency that retrieves the current user’s information based on the provided JWT access token. It first checks for the presence of the token, then decodes and validates it to extract the user ID. It fetches the corresponding user record from the database and returns a `UserDisplayInformation` object containing the user’s details. If any step fails (e.g., missing token, invalid token, user not found), it raises an `HTTPException` with a 401 status code. Used for endpoints that require authentication and need access to the current user’s information.

- **Parameters:** **credentials** (`HTTPAuthorizationCredentials` | `None` , *optional*) – An optional `HTTPAuthorizationCredentials` object provided by the `HTTPBearer` security scheme, containing the access token from the request headers.

- **Returns:** A `UserDisplayInformation` object containing the authenticated user's information if the token is valid and the user exists, or raises an `HTTPException` if authentication fails.
- **Return type:** *UserDisplayInformation*

2.0.5 *async* `require_project_manager_user(project_id: int, credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation`

A FastAPI dependency that ensures the current user has project manager privileges for a specific project. It retrieves the user's information using the provided JWT access token and checks if the user is either an administrator or a project manager for the specified project. If the user does not have the required privileges, it raises an `HTTPException` with a 403 status code. This dependency is used for endpoints that require project manager-level access.

- **Parameters:**
 - **project_id** (*int*) – The ID of the project for which project manager privileges are required.
 - **credentials** (*HTTPAuthorizationCredentials | None, optional*) – An optional `HTTPAuthorizationCredentials` object provided by the `HTTPBearer` security scheme, containing the access token from the request headers.
- **Returns:** A `UserDisplayInformation` object containing the authenticated user's information if the user has project manager privileges for the specified project, or raises an `HTTPException` if the user does not have the required privileges or if authentication fails.
- **Return type:** *UserDisplayInformation*

2.0.6 *async* `require_valid_access_token(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → None`

A FastAPI dependency that checks for the presence of a valid JWT access token in the request headers. It uses the `HTTPBearer` security scheme to extract the token and verifies its validity. If the token is missing, invalid, or expired, it raises an `HTTPException` with a 401 status code. On failure, it raises an `HTTPException` with a 401 status code and an appropriate error message indicating the reason for the failure (e.g., missing token, invalid token, expired token).

- **Parameters:** **credentials** (*HTTPAuthorizationCredentials | None, optional*) – An optional `HTTPAuthorizationCredentials` object provided by the `HTTPBearer` security scheme, containing the access token from the request headers.

2.0.7 `verify_access_token(token: str) → bool`

Verifies if the provided JWT access token is valid by attempting to decode it with the secret key and expected algorithm. It checks for the presence of the required fields and ensures that the token has not expired.

- **Parameters:** `token` (*str*) – The JWT access token string to be verified.
- **Returns:** True if the token is valid and can be decoded successfully, False otherwise (including if the token is expired or malformed).
- **Return type:** *bool*

2.1 Submodules

2.2 `backend.auth.access_token` module

2.2.1 `create_access_token(data: AccessTokenInfo) → str`

Create a JWT access token containing the user information and an expiration time.

- **Parameters:** `data` (*AccessTokenInfo*) – An *AccessTokenInfo* object containing the user ID and optionally the project ID to be included in the token payload.
- **Returns:** A JWT-encoded string that can be used as an access token for authenticating requests to protected endpoints.
- **Return type:** *str*

2.2.2 `async get_user_from_token(token: str, project_id: int | None = None) → UserDisplayInformation | None`

Extract the user information from a JWT access token. It decodes the token, verifies its validity, and retrieves the corresponding user record from the database.

- **Parameters:**
 - `token` (*str*) – The JWT access token string to be decoded and validated.
 - `project_id` (*int | None*, *optional*) – Optional project ID to include permissions in the information.
- **Returns:** A *UserDisplayInformation* object containing the user's information if the token is valid and the user exists, or *None* if the token is invalid or the user cannot be found.
- **Return type:** *UserDisplayInformation | None*

2.2.3 *async* require_administrator_user(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation

A FastAPI dependency that ensures the current user is an administrator. It retrieves the user's information using the provided JWT access token and checks if the user has administrator privileges. If the user is not an administrator, it raises an HTTPException with a 403 status code. This dependency is used for endpoints that require administrator-level access.

- **Parameters:** **credentials** (*HTTPAuthorizationCredentials* | *None* , *optional*) – An optional *HTTPAuthorizationCredentials* object provided by the HTTPBearer security scheme, containing the access token from the request headers.
- **Returns:** A *UserDisplayInformation* object containing the authenticated user's information if the user is an administrator, or raises an HTTPException if the user is not an administrator or if authentication fails.
- **Return type:** *UserDisplayInformation*

2.2.4 *async* require_current_user(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation

A FastAPI dependency that retrieves the current user's information based on the provided JWT access token. It first checks for the presence of the token, then decodes and validates it to extract the user ID. It fetches the corresponding user record from the database and returns a *UserDisplayInformation* object containing the user's details. If any step fails (e.g., missing token, invalid token, user not found), it raises an HTTPException with a 401 status code. Used for endpoints that require authentication and need access to the current user's information.

- **Parameters:** **credentials** (*HTTPAuthorizationCredentials* | *None* , *optional*) – An optional *HTTPAuthorizationCredentials* object provided by the HTTPBearer security scheme, containing the access token from the request headers.
- **Returns:** A *UserDisplayInformation* object containing the authenticated user's information if the token is valid and the user exists, or raises an HTTPException if authentication fails.
- **Return type:** *UserDisplayInformation*

2.2.5 *async* require_project_manager_user(project_id: int, credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → UserDisplayInformation

A FastAPI dependency that ensures the current user has project manager privileges for a specific project. It retrieves the user's information using the provided

JWT access token and checks if the user is either an administrator or a project manager for the specified project. If the user does not have the required privileges, it raises an HTTPException with a 403 status code. This dependency is used for endpoints that require project manager-level access.

- **Parameters:**

- **project_id** (*int*) – The ID of the project for which project manager privileges are required.
- **credentials** (*HTTPAuthorizationCredentials / None , optional*) – An optional HTTPAuthorizationCredentials object provided by the HTTPBearer security scheme, containing the access token from the request headers.

- **Returns:** A UserDisplayInformation object containing the authenticated user's information if the user has project manager privileges for the specified project, or raises an HTTPException if the user does not have the required privileges or if authentication fails.

- **Return type:** *UserDisplayInformation*

2.2.6 `async require_valid_access_token(credentials: HTTPAuthorizationCredentials | None = fastapi.Depends) → None`

A FastAPI dependency that checks for the presence of a valid JWT access token in the request headers. It uses the HTTPBearer security scheme to extract the token and verifies its validity. If the token is missing, invalid, or expired, it raises an HTTPException with a 401 status code. On failure, it raises an HTTPException with a 401 status code and an appropriate error message indicating the reason for the failure (e.g., missing token, invalid token, expired token).

- **Parameters:** **credentials** (*HTTPAuthorizationCredentials / None , optional*) – An optional HTTPAuthorizationCredentials object provided by the HTTPBearer security scheme, containing the access token from the request headers.

2.2.7 `verify_access_token(token: str) → bool`

Verifies if the provided JWT access token is valid by attempting to decode it with the secret key and expected algorithm. It checks for the presence of the required fields and ensures that the token has not expired.

- **Parameters:** **token** (*str*) – The JWT access token string to be verified.
- **Returns:** True if the token is valid and can be decoded successfully, False otherwise (including if the token is expired or malformed).
- **Return type:** *bool*

3 backend.db package

3.0.1 *async* `add_user_to_project(project_id: int, user_id: int, session=None) → None`

Add a user to a project with the default user role.

- **Parameters:**
 - `project_id` (*int*) – Identifier of the project.
 - `user_id` (*int*) – Identifier of the user to add.
 - `session` (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** *None*

3.0.2 *async* `change_user_identity(user_id: int, new_email: str, new_username: str) → User`

Update a user's email and username.

- **Parameters:**
 - `user_id` (*int*) – Identifier of the user.
 - `new_email` (*str*) – New email value.
 - `new_username` (*str*) – New username value.
- **Returns:** The updated User ORM object.
- **Return type:** User

3.0.3 *async* `change_user_password(user_id: int, new_password_hash: str, mandate_reset: bool = False) → User`

Update a user's password hash and optional reset flag timestamp.

- **Parameters:**
 - `user_id` (*int*) – Identifier of the user.
 - `new_password_hash` (*str*) – New hashed password value.
 - `mandate_reset` (*bool*, *optional*) – Whether password expiration should be set immediately.
- **Returns:** The updated User ORM object.
- **Return type:** User

3.0.4 *async* `create_key(project_id: int, user_id: int, name: str, fingerprint: str, key_hash: str, session=None) → APIKey`

Create a new API key record in the database.

- **Parameters:**

- **project_id** (*int*) – Project identifier for the key.
- **user_id** (*int*) – Owner user identifier.
- **name** (*str*) – Human-readable key name.
- **fingerprint** (*str*) – Key fingerprint value.
- **key_hash** (*str*) – Hashed key value.
- **session** (*optional*) – Optional transactional SQLAlchemy session.

- **Returns:** The created APIKey ORM object.

- **Return type:** APIKey

3.0.5 *async* create_project(name: str, manager_id: int, session=None) → Project

Create a project and assign its project manager permission.

- **Parameters:**

- **name** (*str*) – Name for the project.
- **manager_id** (*int*) – Identifier of the initial project manager.
- **session** (*optional*) – Optional transactional SQLAlchemy session.

- **Returns:** The created Project ORM object.

- **Return type:** Project

3.0.6 *async* create_quota(project_id: int | None, user_id: int | None, key_id: int | None, limit_id: int, limit_value: float | None, period: Period, expires_at: datetime | None, session=None)

Create and persist a new quota record.

- **Parameters:**

- **project_id** (*int | None*) – Optional project identifier target.
- **user_id** (*int | None*) – Optional user identifier target.
- **key_id** (*int | None*) – Optional API key identifier target.
- **limit_id** (*int*) – Limit type identifier.
- **limit_value** (*float | None*) – Optional numeric quota limit.
- **period** (*Period*) – Quota period enum value.
- **expires_at** (*datetime | None*) – Optional quota expiration timestamp.
- **session** (*optional*) – Optional transactional SQLAlchemy session.

- **Returns:** The created Quota ORM object.

- **Return type:** Quota

3.0.7 *async* create_user(email: str, username: str, password_hash: str, mandate_reset: bool = False, session=None) → User

Create and persist a new user record.

- **Parameters:**
 - **email** (*str*) – User email address.
 - **username** (*str*) – User username.
 - **password_hash** (*str*) – Hashed password value.
 - **mandate_reset** (*bool* , *optional*) – Whether password expiration is set at creation.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** The created User ORM object.
- **Return type:** User

3.0.8 *async* delete_key(key_id: int, session=None) → None

Archive an API key and delete quotas targeting it.

- **Parameters:**
 - **key_id** (*int*) – Identifier of the API key to archive.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** None

3.0.9 *async* delete_project(project_id: int, session=None) → None

Archive a project and clean related permissions, keys, and quotas.

- **Parameters:**
 - **project_id** (*int*) – Identifier of the project to archive.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** None

3.0.10 *async* delete_quota(quota_id: int, session=None) → None

Delete a quota by identifier.

- **Parameters:**
 - **quota_id** (*int*) – Identifier of the quota to delete.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** None

3.0.11 *async* delete_user(user_id: int, session=None) → None

Delete a user and cleanup related permissions, keys, and quotas.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user to delete.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** *None*

3.0.12 *async* get_all_keys(session=None, active_only: bool = True) → list[APIKey]

Retrieve all API keys across projects.

- **Parameters:**
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.
- **Returns:** A list of APIKey ORM objects.
- **Return type:** *list*[APIKey]

3.0.13 *async* get_all_limits(session=None) → list[Limit]

Retrieve all quota limit types.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Limit ORM objects.
- **Return type:** *list*[Limit]

3.0.14 *async* get_all_projects(session=None) → list[Project]

Retrieve all active projects except the Global project.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Project ORM objects.
- **Return type:** *list*[Project]

3.0.15 *async* get_all_users(session=None) → list[User]

Retrieve all users ordered by username.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of User ORM objects.
- **Return type:** *list*[User]

3.0.16 *async* `get_global_quotas(session=None, include_targeted: bool = False, active_only: bool = False, options=None) → list[Quota]`

Retrieve global quotas, optionally including targeted variants.

- **Parameters:**

- **session** (*optional*) – Optional SQLAlchemy session.
- **include_targeted** (*bool* , *optional*) – Whether targeted global quotas are included.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.

- **Return type:** *list*[Quota]

3.0.17 *async* `get_key_by_id(key_id: int, session=None, options=None, active_only: bool = True) → APIKey | None`

Retrieve an API key by identifier. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**

- **key_id** (*int*) – Identifier of the API key.
- **session** (*optional*) – Optional SQLAlchemy session.
- **options** (*optional*) – Optional relationship loading options.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.

- **Returns:** The matching APIKey ORM object, or None if not found.

- **Return type:** APIKey | None

3.0.18 *async* `get_key_ownership(key_id: int, session=None) → tuple[Project, User]`

Retrieve the project and user owning an API key.

- **Parameters:**

- **key_id** (*int*) – Identifier of the API key.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** A tuple of project and user ORM objects.

- **Return type:** *tuple*[Project, User]

3.0.19 *async* `get_keys_for_project(project_id: int, session=None, active_only: bool = True) → list[APIKey]`

Retrieve all API keys for a project.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.

- **Returns:** A list of APIKey ORM objects.

- **Return type:** *list*[APIKey]

3.0.20 *async* `get_keys_for_user(user_id: int, project_id=None, session=None, active_only: bool = True) → list[APIKey]`

Retrieve all API keys belonging to a user, or the user's keys in a project if a project is provided. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**

- **user_id** (*int*) – Identifier of the user.
- **session** (*optional*) – Optional SQLAlchemy session.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.

- **Returns:** A list of APIKey ORM objects.

- **Return type:** *list*[APIKey]

3.0.21 *async* `get_limit_by_id(limit_id: int, session=None) → Limit | None`

Retrieve a quota limit type by identifier.

- **Parameters:**

- **limit_id** (*int*) – Identifier of the limit type.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** The matching Limit ORM object, or None.

- **Return type:** Limit | None

3.0.22 *async* `get_project_by_id(project_id: int, session=None, options=None, active_only: bool = True) → Project | None`

Retrieve a project by identifier.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.
- **options** (*optional*) – Optional relationship loading options.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active projects.

- **Returns:** The matching Project ORM object, or None.
- **Return type:** Project | None

3.0.23 *async* **get_project_by_name**(project_name: str, session=None, options=None, active_only: bool = True) → Project | None

Retrieve a project by name.

- **Parameters:**
 - **project_name** (*str*) – Name of the project.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active projects.
- **Returns:** The matching Project ORM object, or None.
- **Return type:** Project | None

3.0.24 *async* **get_projects_for_user**(user_id: int, session=None) → list[Project]

Retrieve all active non-global projects assigned to a user.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Project ORM objects.
- **Return type:** list[Project]

3.0.25 *async* **get_quota_by_id**(quota_id: int, session=None, options=None) → Quota | None

Retrieve a quota by identifier.

- **Parameters:**
 - **quota_id** (*int*) – Identifier of the quota.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching Quota ORM object, or None.
- **Return type:** Quota | None

3.0.26 *async* `get_quotas_for_api_key(key_id: int, session=None, include_inherited: bool = True, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for an API key with optional inherited user quotas.

- **Parameters:**

- **key_id** (*int*) – Identifier of the API key.
- **session** (*optional*) – Optional SQLAlchemy session.
- **include_inherited** (*bool* , *optional*) – Whether inherited user quotas are included.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.

- **Return type:** *list*[Quota]

3.0.27 *async* `get_quotas_for_project(project_id: int, session=None, include_targeted: bool = False, include_inherited: bool = True, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for a project with optional inherited global quotas.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.
- **include_targeted** (*bool* , *optional*) – Whether targeted project quotas are included.
- **include_inherited** (*bool* , *optional*) – Whether global quotas are appended.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.

- **Return type:** *list*[Quota]

3.0.28 *async* `get_quotas_for_user(user_id: int, session=None, include_inherited: bool = True, include_keys: bool = False, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for a user with optional inherited and key-level quotas.

- **Parameters:**

- **user_id** (*int*) – Identifier of the user.

- **session** (*optional*) – Optional SQLAlchemy session.
- **include_inherited** (*bool* , *optional*) – Whether global and project quotas are included.
- **include_keys** (*bool* , *optional*) – Whether key-level quotas are included.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.
- **Return type:** *list*[Quota]

3.0.29 *async* **get_role_by_name**(role_name: str, session=None)
→ Role | None

Retrieve a role by name.

- **Parameters:**
 - **role_name** (*str*) – Name of the role.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** The matching Role ORM object, or None.
- **Return type:** Role | None

3.0.30 *async* **get_usage_logs**(user_id: int | None = None, project_id: int | None = None, aggregate_by_fifteen_minutes: bool = False, limit: int | None = None, offset: int = 0, session=None) → list[UsageLog] | list[sqlalchemy.Row]

Retrieve usage logs with optional filtering, pagination, and aggregation.

- **Parameters:**
 - **user_id** (*int* | None , *optional*) – Optional user identifier filter.
 - **project_id** (*int* | None , *optional*) – Optional project identifier filter.
 - **aggregate_by_fifteen_minutes** (*bool* , *optional*) – Whether to return 15-minute aggregate rows.
 - **limit** (*int* | None , *optional*) – Optional maximum number of rows to return.
 - **offset** (*int* , *optional*) – Number of rows to skip.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of UsageLog ORM objects or aggregated SQL rows.
- **Return type:** *list*[UsageLog] | *list*[Row]

3.0.31 *async* `get_user_by_email(email: str, session=None, options=None) → User | None`

Retrieve a user by email address.

- **Parameters:**
 - **email** (*str*) – Email address to search for.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.0.32 *async* `get_user_by_id(user_id: int, session=None, options=None) → User | None`

Retrieve a user by identifier.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.0.33 *async* `get_user_by_username(username: str, session=None, options=None) → User | None`

Retrieve a user by username.

- **Parameters:**
 - **username** (*str*) – Username to search for.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.0.34 *async* `get_user_permissions_for_project(project_id: int, user_id: int, session=None) → ProjectPermission | None`

Retrieve a user's permission record for a project.

- **Parameters:**
 - **project_id** (*int*) – Identifier of the project.
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** ProjectPermission ORM object if found, otherwise None.
- **Return type:** ProjectPermission | None

3.0.35 *async* `get_users_by_project(project_id: int, session=None) → list[User]`

Retrieve users assigned to a project.

- **Parameters:**
 - **project_id** (*int*) – Identifier of the project.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of User ORM objects.
- **Return type:** *list[User]*

3.0.36 *async* `is_password_expired(user_id: int, session=None) → bool`

Check whether a user's password expiration timestamp has passed.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if password is expired, otherwise False.
- **Return type:** *bool*

3.0.37 *async* `is_user_administrator(user_id: int, session=None) → bool`

Check whether a user has global administrator role.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if the user is a global administrator, otherwise False.
- **Return type:** *bool*

3.0.38 *async* `is_user_project_manager(user_id: int, project_id: int | None = None, session=None) → bool`

Check whether a user has project manager role.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **project_id** (*int | None, optional*) – Optional project identifier for project-scoped check.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if the user is a project manager in the requested scope.
- **Return type:** *bool*

3.0.39 *async* `is_user_project_member(project_id: int, user_id: int, session=None) → bool`

Check whether a user is a member of a project.

- **Parameters:**
 - `project_id` (*int*) – Identifier of the project.
 - `user_id` (*int*) – Identifier of the user.
 - `session` (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if membership exists, otherwise False.
- **Return type:** *bool*

3.0.40 *async* `remove_user_from_project(project_id: int, user_id: int, session=None, users_only: bool = True) → None`

Remove a user from a project and clean related project keys and quotas.

- **Parameters:**
 - `project_id` (*int*) – Identifier of the project.
 - `user_id` (*int*) – Identifier of the user to remove.
 - `session` (*optional*) – Optional transactional SQLAlchemy session.
 - `users_only` (*bool* , *optional*) – Whether project managers are protected from removal.
- **Returns:** None.
- **Return type:** *None*

3.0.41 *async* `user_email_free(email: str, exclude_user_id: int | None = None) → bool`

Check whether an email address is available.

- **Parameters:**
 - `email` (*str*) – Email address to validate.
 - `exclude_user_id` (*int* | *None* , *optional*) – Optional user id to ignore in uniqueness checks.
- **Returns:** True if the email can be used, otherwise False.
- **Return type:** *bool*

3.0.42 *async* `user_username_free(username: str, exclude_user_id: int | None = None) → bool`

Check whether a username is available.

- **Parameters:**
 - `username` (*str*) – Username to validate.

- **exclude_user_id** (*int* / *None* , *optional*) – Optional user id to ignore in uniqueness checks.

- **Returns:** True if the username can be used, otherwise False.
- **Return type:** *bool*

3.1 Submodules

3.2 backend.db.keys module

3.2.1 *async* create_key(project_id: int, user_id: int, name: str, fingerprint: str, key_hash: str, session=None) → APIKey

Create a new API key record in the database.

- **Parameters:**
 - **project_id** (*int*) – Project identifier for the key.
 - **user_id** (*int*) – Owner user identifier.
 - **name** (*str*) – Human-readable key name.
 - **fingerprint** (*str*) – Key fingerprint value.
 - **key_hash** (*str*) – Hashed key value.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** The created APIKey ORM object.
- **Return type:** APIKey

3.2.2 *async* delete_key(key_id: int, session=None) → None

Archive an API key and delete quotas targeting it.

- **Parameters:**
 - **key_id** (*int*) – Identifier of the API key to archive.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** *None*

3.2.3 *async* get_all_keys(session=None, active_only: bool = True) → list[APIKey]

Retrieve all API keys across projects.

- **Parameters:**
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.
- **Returns:** A list of APIKey ORM objects.
- **Return type:** *list*[APIKey]

3.2.4 *async* `get_key_by_id(key_id: int, session=None, options=None, active_only: bool = True) → APIKey | None`

Retrieve an API key by identifier. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**

- **key_id** (*int*) – Identifier of the API key.
- **session** (*optional*) – Optional SQLAlchemy session.
- **options** (*optional*) – Optional relationship loading options.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.

- **Returns:** The matching APIKey ORM object, or None if not found.

- **Return type:** APIKey | None

3.2.5 *async* `get_key_ownership(key_id: int, session=None) → tuple[Project, User]`

Retrieve the project and user owning an API key.

- **Parameters:**

- **key_id** (*int*) – Identifier of the API key.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** A tuple of project and user ORM objects.

- **Return type:** tuple[Project, User]

3.2.6 `get_key_relationship_options()`

Return default relationship loading options for API key queries.

3.2.7 *async* `get_keys_for_project(project_id: int, session=None, active_only: bool = True) → list[APIKey]`

Retrieve all API keys for a project.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.

- **Returns:** A list of APIKey ORM objects.

- **Return type:** list[APIKey]

3.2.8 *async* get_keys_for_user(user_id: int, project_id=None, session=None, active_only: bool = True) → list[APIKey]

Retrieve all API keys belonging to a user, or the user's keys in a project if a project is provided. If a session is provided, it uses that session; otherwise, it creates a new session for the query.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active keys and active projects.
- **Returns:** A list of APIKey ORM objects.
- **Return type:** *list*[APIKey]

3.3 backend.db.logs module

3.3.1 *async* get_usage_logs(user_id: int | None = None, project_id: int | None = None, aggregate_by_fifteen_minutes: bool = False, limit: int | None = None, offset: int = 0, session=None) → list[UsageLog] | list[sqlalchemy.Row]

Retrieve usage logs with optional filtering, pagination, and aggregation.

- **Parameters:**
 - **user_id** (*int* | *None* , *optional*) – Optional user identifier filter.
 - **project_id** (*int* | *None* , *optional*) – Optional project identifier filter.
 - **aggregate_by_fifteen_minutes** (*bool* , *optional*) – Whether to return 15-minute aggregate rows.
 - **limit** (*int* | *None* , *optional*) – Optional maximum number of rows to return.
 - **offset** (*int* , *optional*) – Number of rows to skip.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of UsageLog ORM objects or aggregated SQL rows.
- **Return type:** *list*[UsageLog] | *list*[Row]

3.4 backend.db.lookups module

3.4.1 *async* get_all_limits(session=None) → list[Limit]

Retrieve all quota limit types.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Limit ORM objects.
- **Return type:** *list*[Limit]

3.4.2 *async* get_limit_by_id(limit_id: int, session=None) → Limit | None

Retrieve a quota limit type by identifier.

- **Parameters:**
 - **limit_id** (*int*) – Identifier of the limit type.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** The matching Limit ORM object, or None.
- **Return type:** Limit | None

3.4.3 *async* get_user_by_email(email: str, session=None, options=None) → User | None

Retrieve a user by email address.

- **Parameters:**
 - **email** (*str*) – Email address to search for.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.4.4 *async* get_user_by_id(user_id: int, session=None, options=None) → User | None

Retrieve a user by identifier.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.4.5 *async* get_user_by_username(username: str, session=None, options=None) → User | None

Retrieve a user by username.

- **Parameters:**
 - **username** (*str*) – Username to search for.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching User ORM object, or None.
- **Return type:** User | None

3.4.6 `get_user_relationship_options()`

Return default relationship loading options for user queries.

3.4.7 `async get_users_by_project(project_id: int, session=None) → list[User]`

Retrieve users assigned to a project.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** A list of User ORM objects.

- **Return type:** *list[User]*

3.4.8 `async user_email_free(email: str, exclude_user_id: int | None = None) → bool`

Check whether an email address is available.

- **Parameters:**

- **email** (*str*) – Email address to validate.
- **exclude_user_id** (*int | None , optional*) – Optional user id to ignore in uniqueness checks.

- **Returns:** True if the email can be used, otherwise False.

- **Return type:** *bool*

3.4.9 `async user_username_free(username: str, exclude_user_id: int | None = None) → bool`

Check whether a username is available.

- **Parameters:**

- **username** (*str*) – Username to validate.
- **exclude_user_id** (*int | None , optional*) – Optional user id to ignore in uniqueness checks.

- **Returns:** True if the username can be used, otherwise False.

- **Return type:** *bool*

3.5 backend.db.permissions module

3.5.1 `async add_user_to_project(project_id: int, user_id: int, session=None) → None`

Add a user to a project with the default user role.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **user_id** (*int*) – Identifier of the user to add.
- **session** (*optional*) – Optional transactional SQLAlchemy session.

- **Returns:** None.

- **Return type:** *None*

3.5.2 *async* **get_user_permissions_for_project**(project_id: int, user_id: int, session=None) → ProjectPermission | None

Retrieve a user's permission record for a project.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **user_id** (*int*) – Identifier of the user.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** ProjectPermission ORM object if found, otherwise None.

- **Return type:** ProjectPermission | *None*

3.5.3 *async* **is_user_administrator**(user_id: int, session=None) → bool

Check whether a user has global administrator role.

- **Parameters:**

- **user_id** (*int*) – Identifier of the user.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** True if the user is a global administrator, otherwise False.

- **Return type:** *bool*

3.5.4 *async* **is_user_project_manager**(user_id: int, project_id: int | None = None, session=None) → bool

Check whether a user has project manager role.

- **Parameters:**

- **user_id** (*int*) – Identifier of the user.
- **project_id** (*int* | *None* , *optional*) – Optional project identifier for project-scoped check.
- **session** (*optional*) – Optional SQLAlchemy session.

- **Returns:** True if the user is a project manager in the requested scope.

- **Return type:** *bool*

3.5.5 *async* `is_user_project_member(project_id: int, user_id: int, session=None) → bool`

Check whether a user is a member of a project.

- **Parameters:**
 - `project_id` (*int*) – Identifier of the project.
 - `user_id` (*int*) – Identifier of the user.
 - `session` (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if membership exists, otherwise False.
- **Return type:** *bool*

3.5.6 *async* `remove_user_from_project(project_id: int, user_id: int, session=None, users_only: bool = True) → None`

Remove a user from a project and clean related project keys and quotas.

- **Parameters:**
 - `project_id` (*int*) – Identifier of the project.
 - `user_id` (*int*) – Identifier of the user to remove.
 - `session` (*optional*) – Optional transactional SQLAlchemy session.
 - `users_only` (*bool* , *optional*) – Whether project managers are protected from removal.
- **Returns:** None.
- **Return type:** *None*

3.6 backend.db.projects module

3.6.1 *async* `create_project(name: str, manager_id: int, session=None) → Project`

Create a project and assign its project manager permission.

- **Parameters:**
 - `name` (*str*) – Name for the project.
 - `manager_id` (*int*) – Identifier of the initial project manager.
 - `session` (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** The created Project ORM object.
- **Return type:** Project

3.6.2 *async* `delete_project(project_id: int, session=None) → None`

Archive a project and clean related permissions, keys, and quotas.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project to archive.
- **session** (*optional*) – Optional transactional SQLAlchemy session.

- **Returns:** None.
- **Return type:** *None*

3.6.3 *async* get_all_projects(session=None) → list[Project]

Retrieve all active projects except the Global project.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Project ORM objects.
- **Return type:** *list*[Project]

3.6.4 *async* get_project_by_id(project_id: int, session=None, options=None, active_only: bool = True) → Project | None

Retrieve a project by identifier.

- **Parameters:**
 - **project_id** (*int*) – Identifier of the project.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active projects.
- **Returns:** The matching Project ORM object, or None.
- **Return type:** Project | *None*

3.6.5 *async* get_project_by_name(project_name: str, session=None, options=None, active_only: bool = True) → Project | None

Retrieve a project by name.

- **Parameters:**
 - **project_name** (*str*) – Name of the project.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active projects.
- **Returns:** The matching Project ORM object, or None.
- **Return type:** Project | *None*

3.6.6 *async* get_projects_for_user(user_id: int, session=None) → list[Project]

Retrieve all active non-global projects assigned to a user.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of Project ORM objects.
- **Return type:** *list*[Project]

3.7 backend.db.quotas module

3.7.1 *async* create_quota(project_id: int | None, user_id: int | None, key_id: int | None, limit_id: int, limit_value: float | None, period: Period, expires_at: datetime | None, session=None)

Create and persist a new quota record.

- **Parameters:**
 - **project_id** (*int | None*) – Optional project identifier target.
 - **user_id** (*int | None*) – Optional user identifier target.
 - **key_id** (*int | None*) – Optional API key identifier target.
 - **limit_id** (*int*) – Limit type identifier.
 - **limit_value** (*float | None*) – Optional numeric quota limit.
 - **period** (*Period*) – Quota period enum value.
 - **expires_at** (*datetime | None*) – Optional quota expiration timestamp.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** The created Quota ORM object.
- **Return type:** Quota

3.7.2 *async* delete_quota(quota_id: int, session=None) → None

Delete a quota by identifier.

- **Parameters:**
 - **quota_id** (*int*) – Identifier of the quota to delete.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** *None*

3.7.3 *async* `get_global_quotas(session=None, include_targeted: bool = False, active_only: bool = False, options=None) → list[Quota]`

Retrieve global quotas, optionally including targeted variants.

- **Parameters:**
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **include_targeted** (*bool* , *optional*) – Whether targeted global quotas are included.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** A list of Quota ORM objects.
- **Return type:** *list*[Quota]

3.7.4 *async* `get_quota_by_id(quota_id: int, session=None, options=None) → Quota | None`

Retrieve a quota by identifier.

- **Parameters:**
 - **quota_id** (*int*) – Identifier of the quota.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** The matching Quota ORM object, or None.
- **Return type:** Quota | None

3.7.5 *async* `get_quotas_for_api_key(key_id: int, session=None, include_inherited: bool = True, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for an API key with optional inherited user quotas.

- **Parameters:**
 - **key_id** (*int*) – Identifier of the API key.
 - **session** (*optional*) – Optional SQLAlchemy session.
 - **include_inherited** (*bool* , *optional*) – Whether inherited user quotas are included.
 - **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
 - **options** (*optional*) – Optional relationship loading options.
- **Returns:** A list of Quota ORM objects.
- **Return type:** *list*[Quota]

3.7.6 *async* `get_quotas_for_project(project_id: int, session=None, include_targeted: bool = False, include_inherited: bool = True, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for a project with optional inherited global quotas.

- **Parameters:**

- **project_id** (*int*) – Identifier of the project.
- **session** (*optional*) – Optional SQLAlchemy session.
- **include_targeted** (*bool* , *optional*) – Whether targeted project quotas are included.
- **include_inherited** (*bool* , *optional*) – Whether global quotas are appended.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.

- **Return type:** *list*[Quota]

3.7.7 *async* `get_quotas_for_user(user_id: int, session=None, include_inherited: bool = True, include_keys: bool = False, active_only: bool = False, options=None) → list[Quota]`

Retrieve quotas for a user with optional inherited and key-level quotas.

- **Parameters:**

- **user_id** (*int*) – Identifier of the user.
- **session** (*optional*) – Optional SQLAlchemy session.
- **include_inherited** (*bool* , *optional*) – Whether global and project quotas are included.
- **include_keys** (*bool* , *optional*) – Whether key-level quotas are included.
- **active_only** (*bool* , *optional*) – Whether to restrict results to active quotas.
- **options** (*optional*) – Optional relationship loading options.

- **Returns:** A list of Quota ORM objects.

- **Return type:** *list*[Quota]

3.8 backend.db.roles module

3.8.1 *async* `get_role_by_name(role_name: str, session=None) → Role | None`

Retrieve a role by name.

- **Parameters:**
 - **role_name** (*str*) – Name of the role.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** The matching Role ORM object, or None.
- **Return type:** Role | None

3.9 backend.db.users module

3.9.1 *async* **change_user_identity**(user_id: int, new_email: str, new_username: str) → User

Update a user's email and username.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **new_email** (*str*) – New email value.
 - **new_username** (*str*) – New username value.
- **Returns:** The updated User ORM object.
- **Return type:** User

3.9.2 *async* **change_user_password**(user_id: int, new_password_hash: str, mandate_reset: bool = False) → User

Update a user's password hash and optional reset flag timestamp.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **new_password_hash** (*str*) – New hashed password value.
 - **mandate_reset** (*bool* , *optional*) – Whether password expiration should be set immediately.
- **Returns:** The updated User ORM object.
- **Return type:** User

3.9.3 *async* **create_user**(email: str, username: str, password_hash: str, mandate_reset: bool = False, session=None) → User

Create and persist a new user record.

- **Parameters:**
 - **email** (*str*) – User email address.
 - **username** (*str*) – User username.
 - **password_hash** (*str*) – Hashed password value.

- **mandate_reset** (*bool* , *optional*) – Whether password expiration is set at creation.
- **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** The created User ORM object.
- **Return type:** User

3.9.4 *async* delete_user(user_id: int, session=None) → None

Delete a user and cleanup related permissions, keys, and quotas.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user to delete.
 - **session** (*optional*) – Optional transactional SQLAlchemy session.
- **Returns:** None.
- **Return type:** None

3.9.5 *async* get_all_users(session=None) → list[User]

Retrieve all users ordered by username.

- **Parameters:** **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** A list of User ORM objects.
- **Return type:** list[User]

3.9.6 *async* is_password_expired(user_id: int, session=None) → bool

Check whether a user’s password expiration timestamp has passed.

- **Parameters:**
 - **user_id** (*int*) – Identifier of the user.
 - **session** (*optional*) – Optional SQLAlchemy session.
- **Returns:** True if password is expired, otherwise False.
- **Return type:** bool

4 backend.management.keys package

4.0.1 convert_orm_to_display_info(key_orm: APIKey, api_key: str | None = None, username: str | None = None) → ApiKey-DisplayInformation

Convert an API key ORM entity to its dashboard DTO representation.

- **Parameters:**
 - **key_orm** (*APIKey*) – Source API key ORM model.

- **api_key** (*str* | *None* , *optional*) – Optional raw API key value to include in the DTO.
- **username** (*str* | *None* , *optional*) – Optional username override for display.
- **Returns:** *ApiKeyDisplayInformation* built from ORM data.
- **Return type:** *ApiKeyDisplayInformation*

4.0.2 *async* **enforce_key_deletion_permission**(current_user_id: int, key_id: int) → None

Ensure the current user is allowed to delete the target API key.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the user attempting deletion.
 - **key_id** (*int*) – Identifier of the API key to delete.

4.0.3 **generate_api_key**(length: int = 64) → str

Generate a URL-safe random API key string. Based on Pydantic AI Gateway's `generate_api_key.js`.

- **Parameters:** **length** (*int* , *optional*) – Number of random bytes used before Base64 URL-safe encoding.
- **Returns:** A generated API key string.
- **Return type:** *str*

4.1 Submodules

4.2 **backend.management.keys.generation module**

4.2.1 **generate_api_key**(length: int = 64) → str

Generate a URL-safe random API key string. Based on Pydantic AI Gateway's `generate_api_key.js`.

- **Parameters:** **length** (*int* , *optional*) – Number of random bytes used before Base64 URL-safe encoding.
- **Returns:** A generated API key string.
- **Return type:** *str*

4.3 **backend.management.keys.information module**

4.3.1 **convert_orm_to_display_info**(key_orm: *APIKey*, api_key: str | None = None, username: str | None = None) → *ApiKeyDisplayInformation*

Convert an API key ORM entity to its dashboard DTO representation.

- **Parameters:**
 - **key_orm** (*APIKey*) – Source API key ORM model.
 - **api_key** (*str* / *None* , *optional*) – Optional raw API key value to include in the DTO.
 - **username** (*str* / *None* , *optional*) – Optional username override for display.
- **Returns:** *ApiKeyDisplayInformation* built from ORM data.
- **Return type:** *ApiKeyDisplayInformation*

4.4 backend.management.keys.permissions module

4.4.1 *async* enforce_key_deletion_permission(current_user_id: int, key_id: int) → None

Ensure the current user is allowed to delete the target API key.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the user attempting deletion.
 - **key_id** (*int*) – Identifier of the API key to delete.

5 backend.management package

5.0.1 *async* enforce_password_change_validity(current_password: str, current_password_hash: str, new_password: str, new_password_confirm: str) → None

Validate a password change request for correctness and strength.

- **Parameters:**
 - **current_password** (*str*) – Current plain-text password provided by the user.
 - **current_password_hash** (*str*) – Stored password hash for the user.
 - **new_password** (*str*) – Requested new plain-text password.
 - **new_password_confirm** (*str*) – Confirmation of the new password.
- **Returns:** None.
- **Return type:** *None*

5.0.2 *async* enforce_password_strength(password: str) → None

Validate password complexity requirements. A password is considered strong if it is at least 8 characters long and contains at least one uppercase letter, one lowercase letter, and one digit.

- **Parameters:** `password` (*str*) – Password value to validate.
- **Returns:** None.
- **Return type:** *None*

5.0.3 `generate_api_key(length: int = 64) → str`

Generate a URL-safe random API key string. Based on Pydantic AI Gateway's `generate_api_key.js`.

- **Parameters:** `length` (*int* , *optional*) – Number of random bytes used before Base64 URL-safe encoding.
- **Returns:** A generated API key string.
- **Return type:** *str*

5.0.4 `key_convert_orm_to_display_info(key_orm: APIKey, api_key: str | None = None, username: str | None = None) → ApiKeyDisplayInformation`

Convert an API key ORM entity to its dashboard DTO representation.

- **Parameters:**
 - `key_orm` (*APIKey*) – Source API key ORM model.
 - `api_key` (*str* / *None* , *optional*) – Optional raw API key value to include in the DTO.
 - `username` (*str* / *None* , *optional*) – Optional username override for display.
- **Returns:** *ApiKeyDisplayInformation* built from ORM data.
- **Return type:** *ApiKeyDisplayInformation*

5.0.5 `async key_enforce_deletion_permission(current_user_id: int, key_id: int) → None`

Ensure the current user is allowed to delete the target API key.

- **Parameters:**
 - `current_user_id` (*int*) – Identifier of the user attempting deletion.
 - `key_id` (*int*) – Identifier of the API key to delete.

5.0.6 `async log_convert_aggregate_row_to_display_info(row: sqlalchemy.Row) → UsageLogAggregateDisplayInformation`

Convert an aggregated SQL row to an aggregated usage log DTO.

- **Parameters:** `row` (*Row*) – SQLAlchemy Row containing aggregated usage fields.
- **Returns:** *UsageLogAggregateDisplayInformation* mapped from row values.
- **Return type:** *UsageLogAggregateDisplayInformation*

5.0.7 *async* log_convert_orm_to_display_info(usage_log_orm: UsageLog) → UsageLogDisplayInformation

Convert a usage log ORM entity to a usage log display DTO.

- **Parameters:** `usage_log_orm` (*UsageLog*) – Source usage log ORM model.
- **Returns:** *UsageLogDisplayInformation* mapped from ORM data.
- **Return type:** *UsageLogDisplayInformation*

5.0.8 project_convert_orm_to_display_info(project_orm: Project) → ProjectDisplayInfo

Convert a project ORM entity to a project display DTO.

- **Parameters:** `project_orm` (*Project*) – Source project ORM model.
- **Returns:** *ProjectDisplayInfo* mapped from ORM values.
- **Return type:** *ProjectDisplayInfo*

5.0.9 *async* project_enforce_availability(name: str) → None

Ensure a project name is not already used.

- **Parameters:** `name` (*str*) – Project name to validate.
- **Returns:** None.
- **Return type:** *None*

5.0.10 *async* project_enforce_existing_project(project_id: int) → Project

Ensure a project exists and return it.

- **Parameters:** `project_id` (*int*) – Identifier of the project to fetch.
- **Returns:** The found Project ORM entity.
- **Return type:** *Project*

5.0.11 *async* project_enforce_user_not_in_project(user_id: int, project_id: int) → None

Ensure a user is not already a member of the given project.

- **Parameters:**
 - `user_id` (*int*) – Identifier of the user to check.
 - `project_id` (*int*) – Identifier of the target project.
- **Returns:** None.
- **Return type:** *None*

5.0.12 `quota_convert_limit_orm_to_display_info(limit_orm: Limit) → LimitTypeDisplayInformation`

Convert a limit ORM entity to a limit type display DTO.

- **Parameters:** `limit_orm` (*Limit*) – Source limit ORM model.
- **Returns:** `LimitTypeDisplayInformation` mapped from ORM data.
- **Return type:** *LimitTypeDisplayInformation*

5.0.13 `async quota_convert_orm_to_display_info(quota_orm: Quota, add_name: bool = False) → QuotaDisplayInformation`

Convert a quota ORM entity to a quota display DTO.

- **Parameters:**
 - `quota_orm` (*Quota*) – Source quota ORM model.
 - `add_name` (*bool* , *optional*) – Whether a generated human-readable name should be included.
- **Returns:** `QuotaDisplayInformation` mapped from ORM data.
- **Return type:** *QuotaDisplayInformation*

5.0.14 `quota_convert_period_enum_to_display_info(period: Period) → PeriodDisplayInformation`

Convert a period enum value to a period display DTO.

- **Parameters:** `period` (*Period*) – Period enum value.
- **Returns:** `PeriodDisplayInformation` with the enum name.
- **Return type:** *PeriodDisplayInformation*

5.0.15 `async quota_enforce_creation_permission(current_user_id: int, project_id: int | None, user_id: int | None, key_id: int | None) → None`

Ensure the current user can create a quota for the requested scope.

- **Parameters:**
 - `current_user_id` (*int*) – Identifier of the requesting user.
 - `project_id` (*int* / *None*) – Optional project identifier for scoped quotas.
 - `user_id` (*int* / *None*) – Optional user identifier target.
 - `key_id` (*int* / *None*) – Optional API key identifier target.
- **Returns:** `None`.
- **Return type:** *None*

5.0.16 *async* `quota_enforce_deletion_permission(current_user_id: int, quota_id: int) → None`

Ensure the current user can delete the target quota.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the requesting user.
 - **quota_id** (*int*) – Identifier of the quota to delete.
- **Returns:** None.
- **Return type:** *None*

5.0.17 *async* `quota_enforce_existing_limit(limit_id: int) → None`

Ensure a quota limit type definition exists.

- **Parameters:** **limit_id** (*int*) – Identifier of the limit type.
- **Returns:** None.
- **Return type:** *None*

5.0.18 *async* `quota_enforce_existing_quota_target(project_id: int | None, user_id: int | None, key_id: int | None) → None`

Ensure requested quota target entities exist.

- **Parameters:**
 - **project_id** (*int | None*) – Optional project identifier.
 - **user_id** (*int | None*) – Optional user identifier.
 - **key_id** (*int | None*) – Optional API key identifier.
- **Returns:** None.
- **Return type:** *None*

5.0.19 *async* `user_convert_orm_to_display_info(user_orm: User, include_role: bool = False, project_id: int | None = None) → UserDisplayInformation`

Convert a user ORM entity to a user display DTO.

- **Parameters:**
 - **user_orm** (*User*) – Source user ORM model.
 - **include_role** (*bool , optional*) – Whether role information should be resolved and included.
 - **project_id** (*int | None , optional*) – Optional project scope for role resolution.
- **Returns:** UserDisplayInformation mapped from ORM data.
- **Return type:** *UserDisplayInformation*

5.0.20 *async* `user_enforce_availability(email: str, username: str, exclude_user_id: int | None = None) → None`

Ensure email and username values are available for use.

- **Parameters:**
 - **email** (*str*) – Email value to validate.
 - **username** (*str*) – Username value to validate.
 - **exclude_user_id** (*int* / *None* , *optional*) – Optional user identifier to exclude from uniqueness checks.
- **Returns:** None.
- **Return type:** *None*

5.0.21 *async* `user_enforce_existing_user(user_id: int) → User`

Ensure a user exists and return it.

- **Parameters:** `user_id` (*int*) – Identifier of the user to fetch.
- **Returns:** The found User ORM entity.
- **Return type:** `User`

5.1 Subpackages

- backend.management.keys package
 - `convert_orm_to_display_info()`
 - `enforce_key_deletion_permission()`
 - `generate_api_key()`
 - Submodules
 - backend.management.keys.generation module
 - * `generate_api_key()`
 - backend.management.keys.information module
 - * `convert_orm_to_display_info()`
 - backend.management.keys.permissions module
 - * `enforce_key_deletion_permission()`
- backend.management.projects package
 - `convert_orm_to_display_info()`
 - `enforce_existing_project()`
 - `enforce_name_availability()`
 - `enforce_user_not_in_project()`
 - Submodules
 - backend.management.projects.information module
 - * `convert_orm_to_display_info()`
 - * `enforce_existing_project()`

- * enforce_name_availability()
 - * enforce_user_not_in_project()
- backend.management.quotas package
 - convert_limit_orm_to_display_info()
 - convert_orm_to_display_info()
 - convert_period_enum_to_display_info()
 - enforce_existing_limit()
 - enforce_existing_quota_target()
 - enforce_quota_creation_permission()
 - enforce_quota_deletion_permission()
 - Submodules
 - backend.management.quotas.creation module
 - * enforce_existing_limit()
 - * enforce_existing_quota_target()
 - backend.management.quotas.information module
 - * convert_orm_to_display_info()
 - backend.management.quotas.limit_types module
 - * convert_limit_orm_to_display_info()
 - backend.management.quotas.periods module
 - * convert_period_enum_to_display_info()
 - backend.management.quotas.permissions module
 - * enforce_quota_creation_permission()
 - * enforce_quota_deletion_permission()
- backend.management.usage_logs package
 - convert_aggregate_row_to_display_info()
 - convert_orm_to_display_info()
 - Submodules
 - backend.management.usage_logs.information module
 - * convert_aggregate_row_to_display_info()
 - * convert_orm_to_display_info()
- backend.management.users package
 - convert_orm_to_display_info()
 - enforce_availability()
 - enforce_existing_user()
 - enforce_password_change_validity()
 - enforce_password_strength()
 - Submodules
 - backend.management.users.identities module
 - * convert_orm_to_display_info()
 - * enforce_availability()

```

    * enforce_existing_user()
– backend.management.users.passwords module
    * enforce_password_change_validity()
    * enforce_password_strength()

```

6 backend.management.projects package

6.0.1 `convert_orm_to_display_info(project_orm: Project) → ProjectDisplayInfo`

Convert a project ORM entity to a project display DTO.

- **Parameters:** `project_orm` (*Project*) – Source project ORM model.
- **Returns:** `ProjectDisplayInfo` mapped from ORM values.
- **Return type:** *ProjectDisplayInfo*

6.0.2 `async enforce_existing_project(project_id: int) → Project`

Ensure a project exists and return it.

- **Parameters:** `project_id` (*int*) – Identifier of the project to fetch.
- **Returns:** The found `Project` ORM entity.
- **Return type:** `Project`

6.0.3 `async enforce_name_availability(name: str) → None`

Ensure a project name is not already used.

- **Parameters:** `name` (*str*) – Project name to validate.
- **Returns:** `None`.
- **Return type:** *None*

6.0.4 `async enforce_user_not_in_project(user_id: int, project_id: int) → None`

Ensure a user is not already a member of the given project.

- **Parameters:**
 - `user_id` (*int*) – Identifier of the user to check.
 - `project_id` (*int*) – Identifier of the target project.
- **Returns:** `None`.
- **Return type:** *None*

6.1 Submodules

6.2 backend.management.projects.information module

6.2.1 `convert_orm_to_display_info(project_orm: Project) → ProjectDisplayInfo`

Convert a project ORM entity to a project display DTO.

- **Parameters:** `project_orm` (*Project*) – Source project ORM model.
- **Returns:** `ProjectDisplayInfo` mapped from ORM values.
- **Return type:** *ProjectDisplayInfo*

6.2.2 `async enforce_existing_project(project_id: int) → Project`

Ensure a project exists and return it.

- **Parameters:** `project_id` (*int*) – Identifier of the project to fetch.
- **Returns:** The found `Project` ORM entity.
- **Return type:** `Project`

6.2.3 `async enforce_name_availability(name: str) → None`

Ensure a project name is not already used.

- **Parameters:** `name` (*str*) – Project name to validate.
- **Returns:** `None`.
- **Return type:** *None*

6.2.4 `async enforce_user_not_in_project(user_id: int, project_id: int) → None`

Ensure a user is not already a member of the given project.

- **Parameters:**
 - `user_id` (*int*) – Identifier of the user to check.
 - `project_id` (*int*) – Identifier of the target project.
- **Returns:** `None`.
- **Return type:** *None*

7 backend.management.quotas package

7.0.1 `convert_limit_orm_to_display_info(limit_orm: Limit) → LimitTypeDisplayInformation`

Convert a limit ORM entity to a limit type display DTO.

- **Parameters:** `limit_orm` (*Limit*) – Source limit ORM model.
- **Returns:** `LimitTypeDisplayInformation` mapped from ORM data.
- **Return type:** *LimitTypeDisplayInformation*

7.0.2 *async* convert_orm_to_display_info(quota_orm: Quota, add_name: bool = False) → QuotaDisplayInformation

Convert a quota ORM entity to a quota display DTO.

- **Parameters:**
 - **quota_orm** (*Quota*) – Source quota ORM model.
 - **add_name** (*bool* , *optional*) – Whether a generated human-readable name should be included.
- **Returns:** *QuotaDisplayInformation* mapped from ORM data.
- **Return type:** *QuotaDisplayInformation*

7.0.3 convert_period_enum_to_display_info(period: Period) → PeriodDisplayInformation

Convert a period enum value to a period display DTO.

- **Parameters:** **period** (*Period*) – Period enum value.
- **Returns:** *PeriodDisplayInformation* with the enum name.
- **Return type:** *PeriodDisplayInformation*

7.0.4 *async* enforce_existing_limit(limit_id: int) → None

Ensure a quota limit type definition exists.

- **Parameters:** **limit_id** (*int*) – Identifier of the limit type.
- **Returns:** *None*.
- **Return type:** *None*

7.0.5 *async* enforce_existing_quota_target(project_id: int | None, user_id: int | None, key_id: int | None) → None

Ensure requested quota target entities exist.

- **Parameters:**
 - **project_id** (*int* / *None*) – Optional project identifier.
 - **user_id** (*int* / *None*) – Optional user identifier.
 - **key_id** (*int* / *None*) – Optional API key identifier.
- **Returns:** *None*.
- **Return type:** *None*

7.0.6 *async* enforce_quota_creation_permission(current_user_id: int, project_id: int | None, user_id: int | None, key_id: int | None) → None

Ensure the current user can create a quota for the requested scope.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the requesting user.
 - **project_id** (*int* / *None*) – Optional project identifier for scoped quotas.
 - **user_id** (*int* / *None*) – Optional user identifier target.
 - **key_id** (*int* / *None*) – Optional API key identifier target.
- **Returns:** *None*.
- **Return type:** *None*

7.0.7 *async* **enforce_quota_deletion_permission**(current_user_id: int, quota_id: int) → *None*

Ensure the current user can delete the target quota.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the requesting user.
 - **quota_id** (*int*) – Identifier of the quota to delete.
- **Returns:** *None*.
- **Return type:** *None*

7.1 Submodules

7.2 backend.management.quotas.creation module

7.2.1 *async* **enforce_existing_limit**(limit_id: int) → *None*

Ensure a quota limit type definition exists.

- **Parameters:** **limit_id** (*int*) – Identifier of the limit type.
- **Returns:** *None*.
- **Return type:** *None*

7.2.2 *async* **enforce_existing_quota_target**(project_id: int | *None*, user_id: int | *None*, key_id: int | *None*) → *None*

Ensure requested quota target entities exist.

- **Parameters:**
 - **project_id** (*int* / *None*) – Optional project identifier.
 - **user_id** (*int* / *None*) – Optional user identifier.
 - **key_id** (*int* / *None*) – Optional API key identifier.
- **Returns:** *None*.
- **Return type:** *None*

7.3 backend.management.quotas.information module

7.3.1 *async* convert_orm_to_display_info(quota_orm: Quota, add_name: bool = False) → QuotaDisplayInformation

Convert a quota ORM entity to a quota display DTO.

- **Parameters:**
 - **quota_orm** (*Quota*) – Source quota ORM model.
 - **add_name** (*bool* , *optional*) – Whether a generated human-readable name should be included.
- **Returns:** QuotaDisplayInformation mapped from ORM data.
- **Return type:** *QuotaDisplayInformation*

7.4 backend.management.quotas.limit_types module

7.4.1 convert_limit_orm_to_display_info(limit_orm: Limit) → LimitTypeDisplayInformation

Convert a limit ORM entity to a limit type display DTO.

- **Parameters:** **limit_orm** (*Limit*) – Source limit ORM model.
- **Returns:** LimitTypeDisplayInformation mapped from ORM data.
- **Return type:** *LimitTypeDisplayInformation*

7.5 backend.management.quotas.periods module

7.5.1 convert_period_enum_to_display_info(period: Period) → PeriodDisplayInformation

Convert a period enum value to a period display DTO.

- **Parameters:** **period** (*Period*) – Period enum value.
- **Returns:** PeriodDisplayInformation with the enum name.
- **Return type:** *PeriodDisplayInformation*

7.6 backend.management.quotas.permissions module

7.6.1 *async* enforce_quota_creation_permission(current_user_id: int, project_id: int | None, user_id: int | None, key_id: int | None) → None

Ensure the current user can create a quota for the requested scope.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the requesting user.
 - **project_id** (*int* / *None*) – Optional project identifier for scoped quotas.

- **user_id** (*int* / *None*) – Optional user identifier target.
- **key_id** (*int* / *None*) – Optional API key identifier target.
- **Returns:** *None*.
- **Return type:** *None*

7.6.2 *async* **enforce_quota_deletion_permission**(current_user_id: int, quota_id: int) → *None*

Ensure the current user can delete the target quota.

- **Parameters:**
 - **current_user_id** (*int*) – Identifier of the requesting user.
 - **quota_id** (*int*) – Identifier of the quota to delete.
- **Returns:** *None*.
- **Return type:** *None*

8 backend.management.usage_logs package

8.0.1 *async* **convert_aggregate_row_to_display_info**(row: sqlalchemy.Row) → UsageLogAggregateDisplayInformation

Convert an aggregated SQL row to an aggregated usage log DTO.

- **Parameters:** **row** (*Row*) – SQLAlchemy Row containing aggregated usage fields.
- **Returns:** UsageLogAggregateDisplayInformation mapped from row values.
- **Return type:** *UsageLogAggregateDisplayInformation*

8.0.2 *async* **convert_orm_to_display_info**(usage_log_orm: UsageLog) → UsageLogDisplayInformation

Convert a usage log ORM entity to a usage log display DTO.

- **Parameters:** **usage_log_orm** (*UsageLog*) – Source usage log ORM model.
- **Returns:** UsageLogDisplayInformation mapped from ORM data.
- **Return type:** *UsageLogDisplayInformation*

8.1 Submodules

8.2 backend.management.usage_logs.information module

8.2.1 *async* **convert_aggregate_row_to_display_info**(row: sqlalchemy.Row) → UsageLogAggregateDisplayInformation

Convert an aggregated SQL row to an aggregated usage log DTO.

- **Parameters:** `row` (*Row*) – SQLAlchemy Row containing aggregated usage fields.
- **Returns:** `UsageLogAggregateDisplayInformation` mapped from row values.
- **Return type:** *UsageLogAggregateDisplayInformation*

8.2.2 *async* `convert_orm_to_display_info(usage_log_orm: UsageLog) → UsageLogDisplayInformation`

Convert a usage log ORM entity to a usage log display DTO.

- **Parameters:** `usage_log_orm` (*UsageLog*) – Source usage log ORM model.
- **Returns:** `UsageLogDisplayInformation` mapped from ORM data.
- **Return type:** *UsageLogDisplayInformation*

9 backend.management.users package

9.0.1 *async* `convert_orm_to_display_info(user_orm: User, include_role: bool = False, project_id: int | None = None) → UserDisplayInformation`

Convert a user ORM entity to a user display DTO.

- **Parameters:**
 - `user_orm` (*User*) – Source user ORM model.
 - `include_role` (*bool* , *optional*) – Whether role information should be resolved and included.
 - `project_id` (*int* / *None* , *optional*) – Optional project scope for role resolution.
- **Returns:** `UserDisplayInformation` mapped from ORM data.
- **Return type:** *UserDisplayInformation*

9.0.2 *async* `enforce_availability(email: str, username: str, exclude_user_id: int | None = None) → None`

Ensure email and username values are available for use.

- **Parameters:**
 - `email` (*str*) – Email value to validate.
 - `username` (*str*) – Username value to validate.
 - `exclude_user_id` (*int* / *None* , *optional*) – Optional user identifier to exclude from uniqueness checks.
- **Returns:** `None`.
- **Return type:** *None*

9.0.3 *async* enforce_existing_user(user_id: int) → User

Ensure a user exists and return it.

- **Parameters:** `user_id` (*int*) – Identifier of the user to fetch.
- **Returns:** The found User ORM entity.
- **Return type:** User

9.0.4 *async* enforce_password_change_validity(current_password: str, current_password_hash: str, new_password: str, new_password_confirm: str) → None

Validate a password change request for correctness and strength.

- **Parameters:**
 - `current_password` (*str*) – Current plain-text password provided by the user.
 - `current_password_hash` (*str*) – Stored password hash for the user.
 - `new_password` (*str*) – Requested new plain-text password.
 - `new_password_confirm` (*str*) – Confirmation of the new password.
- **Returns:** None.
- **Return type:** *None*

9.0.5 *async* enforce_password_strength(password: str) → None

Validate password complexity requirements. A password is considered strong if it is at least 8 characters long and contains at least one uppercase letter, one lowercase letter, and one digit.

- **Parameters:** `password` (*str*) – Password value to validate.
- **Returns:** None.
- **Return type:** *None*

9.1 Submodules

9.2 backend.management.users.identities module

9.2.1 *async* convert_orm_to_display_info(user_orm: User, include_role: bool = False, project_id: int | None = None) → UserDisplayInformation

Convert a user ORM entity to a user display DTO.

- **Parameters:**
 - `user_orm` (*User*) – Source user ORM model.

- **include_role** (*bool* , *optional*) – Whether role information should be resolved and included.
- **project_id** (*int* / *None* , *optional*) – Optional project scope for role resolution.
- **Returns:** `UserDisplayInformation` mapped from ORM data.
- **Return type:** `UserDisplayInformation`

9.2.2 `async enforce_availability(email: str, username: str, exclude_user_id: int | None = None) → None`

Ensure email and username values are available for use.

- **Parameters:**
 - **email** (*str*) – Email value to validate.
 - **username** (*str*) – Username value to validate.
 - **exclude_user_id** (*int* / *None* , *optional*) – Optional user identifier to exclude from uniqueness checks.
- **Returns:** `None`.
- **Return type:** `None`

9.2.3 `async enforce_existing_user(user_id: int) → User`

Ensure a user exists and return it.

- **Parameters:** **user_id** (*int*) – Identifier of the user to fetch.
- **Returns:** The found User ORM entity.
- **Return type:** `User`

9.3 `backend.management.users.passwords` module

9.3.1 `async enforce_password_change_validity(current_password: str, current_password_hash: str, new_password: str, new_password_confirm: str) → None`

Validate a password change request for correctness and strength.

- **Parameters:**
 - **current_password** (*str*) – Current plain-text password provided by the user.
 - **current_password_hash** (*str*) – Stored password hash for the user.
 - **new_password** (*str*) – Requested new plain-text password.
 - **new_password_confirm** (*str*) – Confirmation of the new password.
- **Returns:** `None`.
- **Return type:** `None`

9.3.2 *async* `enforce_password_strength(password: str) → None`

Validate password complexity requirements. A password is considered strong if it is at least 8 characters long and contains at least one uppercase letter, one lowercase letter, and one digit.

- **Parameters:** `password` (*str*) – Password value to validate.
- **Returns:** `None`.
- **Return type:** *None*

10 backend package

10.1 Subpackages

- `backend.app` package
 - `create_app()`
 - Submodules
 - `backend.app.factory` module
 - * `create_app()`
 - * `lifespan()`
 - * `request_validation_exception_handler()`
 - `backend.app.middleware` module
 - * `init_middleware()`
- `backend.auth` package
 - `create_access_token()`
 - `get_user_from_token()`
 - `require_administrator_user()`
 - `require_current_user()`
 - `require_project_manager_user()`
 - `require_valid_access_token()`
 - `verify_access_token()`
 - Submodules
 - `backend.auth.access_token` module
 - * `create_access_token()`
 - * `get_user_from_token()`
 - * `require_administrator_user()`
 - * `require_current_user()`
 - * `require_project_manager_user()`
 - * `require_valid_access_token()`
 - * `verify_access_token()`
- `backend.db` package
 - `add_user_to_project()`
 - `change_user_identity()`

- change_user_password()
- create_key()
- create_project()
- create_quota()
- create_user()
- delete_key()
- delete_project()
- delete_quota()
- delete_user()
- get_all_keys()
- get_all_limits()
- get_all_projects()
- get_all_users()
- get_global_quotas()
- get_key_by_id()
- get_key_ownership()
- get_keys_for_project()
- get_keys_for_user()
- get_limit_by_id()
- get_project_by_id()
- get_project_by_name()
- get_projects_for_user()
- get_quota_by_id()
- get_quotas_for_api_key()
- get_quotas_for_project()
- get_quotas_for_user()
- get_role_by_name()
- get_usage_logs()
- get_user_by_email()
- get_user_by_id()
- get_user_by_username()
- get_user_permissions_for_project()
- get_users_by_project()
- is_password_expired()
- is_user_administrator()
- is_user_project_manager()
- is_user_project_member()
- remove_user_from_project()
- user_email_free()
- user_username_free()
- Submodules
 - backend.db.keys module
 - * create_key()
 - * delete_key()
 - * get_all_keys()

- * get_key_by_id()
 - * get_key_ownership()
 - * get_key_relationship_options()
 - * get_keys_for_project()
 - * get_keys_for_user()
- backend.db.logs module
 - * get_usage_logs()
- backend.db.lookups module
 - * get_all_limits()
 - * get_limit_by_id()
 - * get_user_by_email()
 - * get_user_by_id()
 - * get_user_by_username()
 - * get_user_relationship_options()
 - * get_users_by_project()
 - * user_email_free()
 - * user_username_free()
- backend.db.permissions module
 - * add_user_to_project()
 - * get_user_permissions_for_project()
 - * is_user_administrator()
 - * is_user_project_manager()
 - * is_user_project_member()
 - * remove_user_from_project()
- backend.db.projects module
 - * create_project()
 - * delete_project()
 - * get_all_projects()
 - * get_project_by_id()
 - * get_project_by_name()
 - * get_projects_for_user()
- backend.db.quotas module
 - * create_quota()
 - * delete_quota()
 - * get_global_quotas()
 - * get_quota_by_id()
 - * get_quotas_for_api_key()
 - * get_quotas_for_project()
 - * get_quotas_for_user()
- backend.db.roles module
 - * get_role_by_name()
- backend.db.users module
 - * change_user_identity()

- * change_user_password()
 - * create_user()
 - * delete_user()
 - * get_all_users()
 - * is_password_expired()
- backend.management package
 - enforce_password_change_validity()
 - enforce_password_strength()
 - generate_api_key()
 - key_convert_orm_to_display_info()
 - key_enforce_deletion_permission()
 - log_convert_aggregate_row_to_display_info()
 - log_convert_orm_to_display_info()
 - project_convert_orm_to_display_info()
 - project_enforce_availability()
 - project_enforce_existing_project()
 - project_enforce_user_not_in_project()
 - quota_convert_limit_orm_to_display_info()
 - quota_convert_orm_to_display_info()
 - quota_convert_period_enum_to_display_info()
 - quota_enforce_creation_permission()
 - quota_enforce_deletion_permission()
 - quota_enforce_existing_limit()
 - quota_enforce_existing_quota_target()
 - user_convert_orm_to_display_info()
 - user_enforce_availability()
 - user_enforce_existing_user()
 - Subpackages
 - * backend.management.keys package
 - convert_orm_to_display_info()
 - enforce_key_deletion_permission()
 - generate_api_key()
 - Submodules
 - backend.management.keys.generation module
 - backend.management.keys.information module
 - backend.management.keys.permissions module
 - * backend.management.projects package
 - convert_orm_to_display_info()
 - enforce_existing_project()
 - enforce_name_availability()
 - enforce_user_not_in_project()
 - Submodules
 - backend.management.projects.information module
 - * backend.management.quotas package

- `convert_limit_orm_to_display_info()`
 - `convert_orm_to_display_info()`
 - `convert_period_enum_to_display_info()`
 - `enforce_existing_limit()`
 - `enforce_existing_quota_target()`
 - `enforce_quota_creation_permission()`
 - `enforce_quota_deletion_permission()`
 - Submodules
 - `backend.management.quotas.creation` module
 - `backend.management.quotas.information` module
 - `backend.management.quotas.limit_types` module
 - `backend.management.quotas.periods` module
 - `backend.management.quotas.permissions` module
- * `backend.management.usage_logs` package
 - `convert_aggregate_row_to_display_info()`
 - `convert_orm_to_display_info()`
 - Submodules
 - `backend.management.usage_logs.information` module
- * `backend.management.users` package
 - `convert_orm_to_display_info()`
 - `enforce_availability()`
 - `enforce_existing_user()`
 - `enforce_password_change_validity()`
 - `enforce_password_strength()`
 - Submodules
 - `backend.management.users.identities` module
 - `backend.management.users.passwords` module
- `backend.models` package
 - `AccessTokenInfo`
 - * `AccessTokenInfo.sub`
 - * `AccessTokenInfo.user_id`
 - `AddUserToProjectRequest`
 - * `AddUserToProjectRequest.project_id`
 - * `AddUserToProjectRequest.user_id`
 - `ApiKeyDeleteRequest`
 - * `ApiKeyDeleteRequest.key_id`
 - `ApiKeyDisplayInformation`
 - * `ApiKeyDisplayInformation.api_key`
 - * `ApiKeyDisplayInformation.create_date`
 - * `ApiKeyDisplayInformation.fingerprint`
 - * `ApiKeyDisplayInformation.id`
 - * `ApiKeyDisplayInformation.name`

- * ApiKeyDisplayInformation.project_id
 - * ApiKeyDisplayInformation.status
 - * ApiKeyDisplayInformation.user_id
 - * ApiKeyDisplayInformation.username
- CreateApiKeyRequest
 - * CreateApiKeyRequest.name
 - * CreateApiKeyRequest.project_id
- CreateProjectRequest
 - * CreateProjectRequest.manager_id
 - * CreateProjectRequest.name
- KeyInformationRequest
 - * KeyInformationRequest.project_id
 - * KeyInformationRequest.user_id
- LimitTypeDisplayInformation
 - * LimitTypeDisplayInformation.description
 - * LimitTypeDisplayInformation.id
 - * LimitTypeDisplayInformation.name
- LoginRequest
 - * LoginRequest.email
 - * LoginRequest.password
- PeriodDisplayInformation
 - * PeriodDisplayInformation.name
- ProjectDeleteRequest
 - * ProjectDeleteRequest.project_id
- ProjectDisplayInfo
 - * ProjectDisplayInfo.created_date
 - * ProjectDisplayInfo.id
 - * ProjectDisplayInfo.modified_date
 - * ProjectDisplayInfo.name
 - * ProjectDisplayInfo.status
- QuotaCreateRequest
 - * QuotaCreateRequest.expires_at
 - * QuotaCreateRequest.key_id
 - * QuotaCreateRequest.limit_id
 - * QuotaCreateRequest.limit_value
 - * QuotaCreateRequest.period
 - * QuotaCreateRequest.project_id
 - * QuotaCreateRequest.user_id
 - * QuotaCreateRequest.validate_request()
- QuotaDeleteRequest
 - * QuotaDeleteRequest.id

- QuotaDisplayInformation
 - * QuotaDisplayInformation.allocated
 - * QuotaDisplayInformation.expires_at
 - * QuotaDisplayInformation.fingerprint
 - * QuotaDisplayInformation.id
 - * QuotaDisplayInformation.limit_id
 - * QuotaDisplayInformation.limit_name
 - * QuotaDisplayInformation.limit_value
 - * QuotaDisplayInformation.name
 - * QuotaDisplayInformation.next_reset
 - * QuotaDisplayInformation.period
 - * QuotaDisplayInformation.project_id
 - * QuotaDisplayInformation.status
 - * QuotaDisplayInformation.user_name
- QuotaInformationRequest
 - * QuotaInformationRequest.active_only
 - * QuotaInformationRequest.include_inherited
 - * QuotaInformationRequest.key_id
 - * QuotaInformationRequest.project_id
 - * QuotaInformationRequest.user_id
 - * QuotaInformationRequest.validate_request()
- TokenResponse
 - * TokenResponse.access_token
 - * TokenResponse.token_type
- UsageLogAggregateDisplayInformation
 - * UsageLogAggregateDisplayInformation.fingerprint
 - * UsageLogAggregateDisplayInformation.internal_cost_final
 - * UsageLogAggregateDisplayInformation.project_name
 - * UsageLogAggregateDisplayInformation.request_count
 - * UsageLogAggregateDisplayInformation.timestamp
 - * UsageLogAggregateDisplayInformation.total_tokens
 - * UsageLogAggregateDisplayInformation.user_name
- UsageLogDisplayInformation
 - * UsageLogDisplayInformation.completion_tokens
 - * UsageLogDisplayInformation.estimated_tokens
 - * UsageLogDisplayInformation.fingerprint
 - * UsageLogDisplayInformation.finish_reason
 - * UsageLogDisplayInformation.gateway_overhead
 - * UsageLogDisplayInformation.id
 - * UsageLogDisplayInformation.internal_cost_estimate
 - * UsageLogDisplayInformation.internal_cost_final
 - * UsageLogDisplayInformation.is_streaming
 - * UsageLogDisplayInformation.model
 - * UsageLogDisplayInformation.project_name

- * UsageLogDisplayInformation.prompt_tokens
- * UsageLogDisplayInformation.request_id
- * UsageLogDisplayInformation.request_type
- * UsageLogDisplayInformation.status_code
- * UsageLogDisplayInformation.temperature
- * UsageLogDisplayInformation.timestamp
- * UsageLogDisplayInformation.top_k
- * UsageLogDisplayInformation.top_p
- * UsageLogDisplayInformation.total_latency
- * UsageLogDisplayInformation.total_tokens
- * UsageLogDisplayInformation.ttft
- * UsageLogDisplayInformation.upstream_latency
- * UsageLogDisplayInformation.user_name
- UsageLogInformationRequest
 - * UsageLogInformationRequest.aggregate
 - * UsageLogInformationRequest.limit
 - * UsageLogInformationRequest.offset
 - * UsageLogInformationRequest.project_id
 - * UsageLogInformationRequest.user_id
- UserDeleteRequest
 - * UserDeleteRequest.user_id
- UserDisplayInformation
 - * UserDisplayInformation.email
 - * UserDisplayInformation.id
 - * UserDisplayInformation.is_admin
 - * UserDisplayInformation.is_password_expired
 - * UserDisplayInformation.is_project_manager
 - * UserDisplayInformation.joined_date
 - * UserDisplayInformation.last_login
 - * UserDisplayInformation.password_expires_at
 - * UserDisplayInformation.profile_picture_url
 - * UserDisplayInformation.role
 - * UserDisplayInformation.username
- UserIdentityChangeRequest
 - * UserIdentityChangeRequest.email
 - * UserIdentityChangeRequest.username
- UserInformationRequest
 - * UserInformationRequest.project_id
- UserPasswordChangeRequest
 - * UserPasswordChangeRequest.current_password
 - * UserPasswordChangeRequest.mandate_reset
 - * UserPasswordChangeRequest.new_password
 - * UserPasswordChangeRequest.new_password_confirm

- * UserPasswordChangeRequest.user_id
- UserRegistrationRequest
 - * UserRegistrationRequest.email
 - * UserRegistrationRequest.mandate_reset
 - * UserRegistrationRequest.password
 - * UserRegistrationRequest.username
- Subpackages
 - * backend.models.dto_models package
 - AccessTokenInfo
 - AddUserToProjectRequest
 - ApiKeyDeleteRequest
 - ApiKeyDisplayInformation
 - CreateApiKeyRequest
 - CreateProjectRequest
 - KeyInformationRequest
 - LimitTypeDisplayInformation
 - LoginRequest
 - PeriodDisplayInformation
 - ProjectDeleteRequest
 - ProjectDisplayInfo
 - QuotaCreateRequest
 - QuotaDeleteRequest
 - QuotaDisplayInformation
 - QuotaInformationRequest
 - TokenResponse
 - UsageLogAggregateDisplayInformation
 - UsageLogDisplayInformation
 - UsageLogInformationRequest
 - UserDeleteRequest
 - UserDisplayInformation
 - UserIdentityChangeRequest
 - UserInformationRequest
 - UserPasswordChangeRequest
 - UserRegistrationRequest
 - Submodules
 - backend.models.dto_models.key module
 - backend.models.dto_models.login module
 - backend.models.dto_models.logs module
 - backend.models.dto_models.project module
 - backend.models.dto_models.quotas module
 - backend.models.dto_models.user module
- backend.routes package
 - Subpackages
 - * backend.routes.auth package

- Submodules
- backend.routes.auth.login module
- backend.routes.auth.register module
- * backend.routes.keys package
 - Submodules
 - backend.routes.keys.create module
 - backend.routes.keys.delete module
 - backend.routes.keys.info module
- * backend.routes.projects package
 - Submodules
 - backend.routes.projects.add_user module
 - backend.routes.projects.all module
 - backend.routes.projects.create module
 - backend.routes.projects.delete module
 - backend.routes.projects.info module
 - backend.routes.projects.remove_user module
- * backend.routes.quotas package
 - Submodules
 - backend.routes.quotas.create module
 - backend.routes.quotas.delete module
 - backend.routes.quotas.info module
 - backend.routes.quotas.limit_types module
 - backend.routes.quotas.periods module
- * backend.routes.usage_logs package
 - Submodules
 - backend.routes.usage_logs.info module
- * backend.routes.users package
 - Submodules
 - backend.routes.users.change_identity module
 - backend.routes.users.change_password module
 - backend.routes.users.delete module
 - backend.routes.users.everyone module
 - backend.routes.users.info module
 - backend.routes.users.me module
- Submodules
- backend.routes.health module
 - * health()
 - * root()

11 backend.models.dto_models package

11.0.1 class AccessTokenInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO payload stored in generated access tokens.

sub : *pydantic.networks.EmailStr*

user_id : *int*

11.0.2 *class* AddUserToProjectRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for adding or removing a user in a project.

project_id : *int*

user_id : *int*

11.0.3 *class* ApiKeyDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting or archiving an API key by identifier.

key_id : *int*

11.0.4 *class* ApiKeyDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing API key information returned to clients.

api_key : *str / None = None*

create_date : *datetime*

fingerprint : *str*

id : *int*

name : *str*

project_id : *int*

status : *Status*

user_id : *int*

username : *str* / *None* = *None*

11.0.5 *class* CreateApiKeyRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a new API key in a project.

name : *str*

project_id : *int*

11.0.6 *class* CreateProjectRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a new project.

manager_id : *int*

name : *str*

11.0.7 *class* KeyInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying API keys by project or user scope.

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

11.0.8 *class* LimitTypeDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing an available quota limit type.

description : *str*

id : *int*

name : *str*

11.0.9 *class* LoginRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for user login credentials.

email : *pydantic.networks.EmailStr*

password : *str*

11.0.10 *class* PeriodDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing a supported quota period value.

name : *str*

11.0.11 *class* ProjectDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for archiving a project by identifier.

project_id : *int*

11.0.12 *class* ProjectDisplayInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing project details returned by project endpoints.

created_date : *datetime*

id : *int*

modified_date : *datetime / None = None*

name : *str*

status : *Status*

11.0.13 *class* QuotaCreateRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a quota for a project, user, or API key.

expires_at : *datetime / None = None*

key_id : *int / None = None*

limit_id : *int*

limit_value : float / None = None

period : Period

project_id : int / None = None

user_id : int / None = None

validate_request()

11.0.14 *class* QuotaDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting a quota by identifier.

id : int

11.0.15 *class* QuotaDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing quota information returned by dashboard endpoints.

allocated : float

expires_at : datetime / None = None

fingerprint : str / None = None

id : int

limit_id : int

limit_name : str

limit_value : float / None = None

name : str / None = None

next_reset : datetime

period : str

project_id : int / None = None

status : *Status*

user_name : *str* / *None* = *None*

11.0.16 *class* **QuotaInformationRequest**(*args: Any, **kwargs: Any)

Bases: **BaseModel**

DTO for querying quotas by optional scope and filter flags.

active_only : *bool* = *False*

include_inherited : *bool* = *True*

key_id : *int* / *None* = *None*

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

validate_request()

11.0.17 *class* **TokenResponse**(*args: Any, **kwargs: Any)

Bases: **BaseModel**

DTO returned after successful authentication.

access_token : *str*

token_type : *str* = *'bearer'*

11.0.18 *class* **UsageLogAggregateDisplayInformation**(*args: Any, **kwargs: Any)

Bases: **BaseModel**

DTO representing aggregated usage metrics for a time window.

fingerprint : *str*

internal_cost_final : *Decimal* / *None* = *None*

project_name : *str*

request_count : *int*

timestamp : datetime

total_tokens : int / None = None

user_name : str

11.0.19 *class UsageLogDisplayInformation(*args: Any, **kwargs: Any)*

Bases: BaseModel

DTO representing a single usage log entry returned to clients.

completion_tokens : int / None = None

estimated_tokens : int / None = None

fingerprint : str

finish_reason : str / None = None

gateway_overhead : float / None = None

id : int

internal_cost_estimate : Decimal / None = None

internal_cost_final : Decimal / None = None

is_streaming : bool / None = None

model : str / None = None

project_name : str

prompt_tokens : int / None = None

request_id : str / None = None

request_type : str / None = None

status_code : int / None = None

temperature : float / None = None

timestamp : datetime

top_k : int / None = None

top_p : float / None = None

total_latency : float / None = None

total_tokens : int / None = None

ttft : float / None = None

upstream_latency : float / None = None

user_name : str

11.0.20 *class* UsageLogInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying usage logs by scope, pagination, and aggregation mode.

aggregate : bool = False

limit : int / None = None

offset : int = 0

project_id : int / None = None

user_id : int / None = None

11.0.21 *class* UserDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting a user by identifier.

user_id : int

11.0.22 *class* UserDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing user information returned by dashboard endpoints.

email : pydantic.networks.EmailStr

id : int

is_admin : bool = False

is_password_expired : bool = False

is_project_manager : bool = False

joined_date : datetime

last_login : datetime / None = None

password_expires_at : datetime / None = None

profile_picture_url : str / None = None

role : str / None = None

username : str

11.0.23 *class* `UserIdentityChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's email and username.

email : pydantic.networks.EmailStr

username : str

11.0.24 *class* `UserInformationRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for requesting user information with optional project scope.

project_id : int / None = None

11.0.25 *class* `UserPasswordChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's password.

current_password : str / None = None

mandate_reset : bool / None = False

new_password : str

new_password_confirm : str

user_id : int / None = None

11.0.26 *class* UserRegistrationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for registering a new user.

email : pydantic.networks.EmailStr

mandate_reset : bool = False

password : str

username : str

11.1 Submodules

11.2 backend.models.dto_models.key module

11.2.1 *class* ApiKeyDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting or archiving an API key by identifier.

key_id : int

11.2.2 *class* ApiKeyDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing API key information returned to clients.

api_key : str / None = None

create_date : datetime

fingerprint : str

id : *int*

name : *str*

project_id : *int*

status : *Status*

user_id : *int*

username : *str / None = None*

11.2.3 *class* CreateApiKeyRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a new API key in a project.

name : *str*

project_id : *int*

11.2.4 *class* KeyInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying API keys by project or user scope.

project_id : *int / None = None*

user_id : *int / None = None*

11.3 backend.models.dto_models.login module

11.3.1 *class* AccessTokenInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO payload stored in generated access tokens.

sub : *pydantic.networks.EmailStr*

user_id : *int*

11.3.2 *class* LoginRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for user login credentials.

email : pydantic.networks.EmailStr

password : str

11.3.3 class TokenResponse(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO returned after successful authentication.

access_token : str

token_type : str = 'bearer'

11.4 backend.models.dto_models.logs module

11.4.1 class UsageLogAggregateDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing aggregated usage metrics for a time window.

fingerprint : str

internal_cost_final : Decimal / None = None

project_name : str

request_count : int

timestamp : datetime

total_tokens : int / None = None

user_name : str

11.4.2 class UsageLogDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing a single usage log entry returned to clients.

completion_tokens : int / None = None

estimated_tokens : int / None = None

`fingerprint : str`

`finish_reason : str / None = None`

`gateway_overhead : float / None = None`

`id : int`

`internal_cost_estimate : Decimal / None = None`

`internal_cost_final : Decimal / None = None`

`is_streaming : bool / None = None`

`model : str / None = None`

`project_name : str`

`prompt_tokens : int / None = None`

`request_id : str / None = None`

`request_type : str / None = None`

`status_code : int / None = None`

`temperature : float / None = None`

`timestamp : datetime`

`top_k : int / None = None`

`top_p : float / None = None`

`total_latency : float / None = None`

`total_tokens : int / None = None`

`ttft : float / None = None`

`upstream_latency : float / None = None`

user_name : *str*

11.4.3 class UsageLogInformationRequest(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO for querying usage logs by scope, pagination, and aggregation mode.

aggregate : *bool* = *False*

limit : *int* / *None* = *None*

offset : *int* = *0*

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

11.5 backend.models.dto_models.project module

11.5.1 class AddUserToProjectRequest(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO for adding or removing a user in a project.

project_id : *int*

user_id : *int*

11.5.2 class CreateProjectRequest(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO for creating a new project.

manager_id : *int*

name : *str*

11.5.3 class ProjectDeleteRequest(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO for archiving a project by identifier.

project_id : *int*

11.5.4 *class* ProjectDisplayInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing project details returned by project endpoints.

created_date : datetime

id : int

modified_date : datetime / None = None

name : str

status : Status

11.6 backend.models.dto_models.quotas module

11.6.1 *class* LimitTypeDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing an available quota limit type.

description : str

id : int

name : str

11.6.2 *class* PeriodDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing a supported quota period value.

name : str

11.6.3 *class* QuotaCreateRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a quota for a project, user, or API key.

expires_at : datetime / None = None

key_id : int / None = None

limit_id : *int*

limit_value : *float / None = None*

period : *Period*

project_id : *int / None = None*

user_id : *int / None = None*

validate_request()

11.6.4 class QuotaDeleteRequest(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO for deleting a quota by identifier.

id : *int*

11.6.5 class QuotaDisplayInformation(*args: Any, **kwargs: Any)

Bases: `BaseModel`

DTO representing quota information returned by dashboard endpoints.

allocated : *float*

expires_at : *datetime / None = None*

fingerprint : *str / None = None*

id : *int*

limit_id : *int*

limit_name : *str*

limit_value : *float / None = None*

name : *str / None = None*

next_reset : *datetime*

period : *str*

project_id : *int* / *None* = *None*

status : *Status*

user_name : *str* / *None* = *None*

11.6.6 class QuotaInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying quotas by optional scope and filter flags.

active_only : *bool* = *False*

include_inherited : *bool* = *True*

key_id : *int* / *None* = *None*

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

validate_request()

11.7 backend.models.dto_models.user module

11.7.1 class UserDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting a user by identifier.

user_id : *int*

11.7.2 class UserDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing user information returned by dashboard endpoints.

email : *pydantic.networks.EmailStr*

id : *int*

is_admin : *bool* = *False*

is_password_expired : *bool* = *False*

is_project_manager : bool = False

joined_date : datetime

last_login : datetime / None = None

password_expires_at : datetime / None = None

profile_picture_url : str / None = None

role : str / None = None

username : str

11.7.3 class `UserIdentityChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's email and username.

email : pydantic.networks.EmailStr

username : str

11.7.4 class `UserInformationRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for requesting user information with optional project scope.

project_id : int / None = None

11.7.5 class `UserPasswordChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's password.

current_password : str / None = None

mandate_reset : bool / None = False

new_password : str

new_password_confirm : str

user_id : int / None = None

11.7.6 *class* UserRegistrationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for registering a new user.

email : pydantic.networks.EmailStr

mandate_reset : bool = False

password : str

username : str

12 backend.models package

12.0.1 *class* AccessTokenInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO payload stored in generated access tokens.

sub : pydantic.networks.EmailStr

user_id : int

12.0.2 *class* AddUserToProjectRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for adding or removing a user in a project.

project_id : int

user_id : int

12.0.3 *class* ApiKeyDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting or archiving an API key by identifier.

key_id : int

12.0.4 *class* ApiKeyDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing API key information returned to clients.

api_key : str / None = None

create_date : datetime

fingerprint : str

id : int

name : str

project_id : int

status : Status

user_id : int

username : str / None = None

12.0.5 *class* CreateApiKeyRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a new API key in a project.

name : str

project_id : int

12.0.6 *class* CreateProjectRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a new project.

manager_id : int

name : str

12.0.7 *class* KeyInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying API keys by project or user scope.

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

12.0.8 *class* LimitTypeDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing an available quota limit type.

description : *str*

id : *int*

name : *str*

12.0.9 *class* LoginRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for user login credentials.

email : *pydantic.networks.EmailStr*

password : *str*

12.0.10 *class* PeriodDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing a supported quota period value.

name : *str*

12.0.11 *class* ProjectDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for archiving a project by identifier.

project_id : *int*

12.0.12 *class* ProjectDisplayInfo(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing project details returned by project endpoints.

created_date : datetime

id : int

modified_date : datetime / None = None

name : str

status : Status

12.0.13 *class* QuotaCreateRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for creating a quota for a project, user, or API key.

expires_at : datetime / None = None

key_id : int / None = None

limit_id : int

limit_value : float / None = None

period : Period

project_id : int / None = None

user_id : int / None = None

validate_request()

12.0.14 *class* QuotaDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting a quota by identifier.

id : int

12.0.15 *class* QuotaDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing quota information returned by dashboard endpoints.

allocated : *float*

expires_at : *datetime / None = None*

fingerprint : *str / None = None*

id : *int*

limit_id : *int*

limit_name : *str*

limit_value : *float / None = None*

name : *str / None = None*

next_reset : *datetime*

period : *str*

project_id : *int / None = None*

status : *Status*

user_name : *str / None = None*

12.0.16 *class* QuotaInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying quotas by optional scope and filter flags.

active_only : *bool = False*

include_inherited : *bool = True*

key_id : *int / None = None*

`project_id : int / None = None`

`user_id : int / None = None`

`validate_request()`

12.0.17 `class TokenResponse(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO returned after successful authentication.

`access_token : str`

`token_type : str = 'bearer'`

12.0.18 `class UsageLogAggregateDisplayInformation(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO representing aggregated usage metrics for a time window.

`fingerprint : str`

`internal_cost_final : Decimal / None = None`

`project_name : str`

`request_count : int`

`timestamp : datetime`

`total_tokens : int / None = None`

`user_name : str`

12.0.19 `class UsageLogDisplayInformation(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO representing a single usage log entry returned to clients.

`completion_tokens : int / None = None`

`estimated_tokens : int / None = None`

`fingerprint : str`

`finish_reason : str / None = None`

`gateway_overhead : float / None = None`

`id : int`

`internal_cost_estimate : Decimal / None = None`

`internal_cost_final : Decimal / None = None`

`is_streaming : bool / None = None`

`model : str / None = None`

`project_name : str`

`prompt_tokens : int / None = None`

`request_id : str / None = None`

`request_type : str / None = None`

`status_code : int / None = None`

`temperature : float / None = None`

`timestamp : datetime`

`top_k : int / None = None`

`top_p : float / None = None`

`total_latency : float / None = None`

`total_tokens : int / None = None`

`ttft : float / None = None`

`upstream_latency : float / None = None`

user_name : *str*

12.0.20 *class* UsageLogInformationRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for querying usage logs by scope, pagination, and aggregation mode.

aggregate : *bool* = *False*

limit : *int* / *None* = *None*

offset : *int* = *0*

project_id : *int* / *None* = *None*

user_id : *int* / *None* = *None*

12.0.21 *class* UserDeleteRequest(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO for deleting a user by identifier.

user_id : *int*

12.0.22 *class* UserDisplayInformation(*args: Any, **kwargs: Any)

Bases: BaseModel

DTO representing user information returned by dashboard endpoints.

email : *pydantic.networks.EmailStr*

id : *int*

is_admin : *bool* = *False*

is_password_expired : *bool* = *False*

is_project_manager : *bool* = *False*

joined_date : *datetime*

last_login : *datetime* / *None* = *None*

`password_expires_at : datetime / None = None`

`profile_picture_url : str / None = None`

`role : str / None = None`

`username : str`

12.0.23 `class UserIdentityChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's email and username.

`email : pydantic.networks.EmailStr`

`username : str`

12.0.24 `class UserInformationRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for requesting user information with optional project scope.

`project_id : int / None = None`

12.0.25 `class UserPasswordChangeRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for changing a user's password.

`current_password : str / None = None`

`mandate_reset : bool / None = False`

`new_password : str`

`new_password_confirm : str`

`user_id : int / None = None`

12.0.26 `class UserRegistrationRequest(*args: Any, **kwargs: Any)`

Bases: `BaseModel`

DTO for registering a new user.

email : pydantic.networks.EmailStr

mandate_reset : bool = False

password : str

username : str

12.1 Subpackages

- backend.models.dto_models package
 - AccessTokenInfo
 - * AccessTokenInfo.sub
 - * AccessTokenInfo.user_id
 - AddUserToProjectRequest
 - * AddUserToProjectRequest.project_id
 - * AddUserToProjectRequest.user_id
 - ApiKeyDeleteRequest
 - * ApiKeyDeleteRequest.key_id
 - ApiKeyDisplayInformation
 - * ApiKeyDisplayInformation.api_key
 - * ApiKeyDisplayInformation.create_date
 - * ApiKeyDisplayInformation.fingerprint
 - * ApiKeyDisplayInformation.id
 - * ApiKeyDisplayInformation.name
 - * ApiKeyDisplayInformation.project_id
 - * ApiKeyDisplayInformation.status
 - * ApiKeyDisplayInformation.user_id
 - * ApiKeyDisplayInformation.username
 - CreateApiKeyRequest
 - * CreateApiKeyRequest.name
 - * CreateApiKeyRequest.project_id
 - CreateProjectRequest
 - * CreateProjectRequest.manager_id
 - * CreateProjectRequest.name
 - KeyInformationRequest
 - * KeyInformationRequest.project_id
 - * KeyInformationRequest.user_id
 - LimitTypeDisplayInformation
 - * LimitTypeDisplayInformation.description
 - * LimitTypeDisplayInformation.id

- * LimitTypeDisplayInformation.name
- LoginRequest
 - * LoginRequest.email
 - * LoginRequest.password
- PeriodDisplayInformation
 - * PeriodDisplayInformation.name
- ProjectDeleteRequest
 - * ProjectDeleteRequest.project_id
- ProjectDisplayInfo
 - * ProjectDisplayInfo.created_date
 - * ProjectDisplayInfo.id
 - * ProjectDisplayInfo.modified_date
 - * ProjectDisplayInfo.name
 - * ProjectDisplayInfo.status
- QuotaCreateRequest
 - * QuotaCreateRequest.expires_at
 - * QuotaCreateRequest.key_id
 - * QuotaCreateRequest.limit_id
 - * QuotaCreateRequest.limit_value
 - * QuotaCreateRequest.period
 - * QuotaCreateRequest.project_id
 - * QuotaCreateRequest.user_id
 - * QuotaCreateRequest.validate_request()
- QuotaDeleteRequest
 - * QuotaDeleteRequest.id
- QuotaDisplayInformation
 - * QuotaDisplayInformation.allocated
 - * QuotaDisplayInformation.expires_at
 - * QuotaDisplayInformation.fingerprint
 - * QuotaDisplayInformation.id
 - * QuotaDisplayInformation.limit_id
 - * QuotaDisplayInformation.limit_name
 - * QuotaDisplayInformation.limit_value
 - * QuotaDisplayInformation.name
 - * QuotaDisplayInformation.next_reset
 - * QuotaDisplayInformation.period
 - * QuotaDisplayInformation.project_id
 - * QuotaDisplayInformation.status
 - * QuotaDisplayInformation.user_name
- QuotaInformationRequest
 - * QuotaInformationRequest.active_only
 - * QuotaInformationRequest.include_inherited

- * QuotaInformationRequest.key_id
- * QuotaInformationRequest.project_id
- * QuotaInformationRequest.user_id
- * QuotaInformationRequest.validate_request()
- TokenResponse
 - * TokenResponse.access_token
 - * TokenResponse.token_type
- UsageLogAggregateDisplayInformation
 - * UsageLogAggregateDisplayInformation.fingerprint
 - * UsageLogAggregateDisplayInformation.internal_cost_final
 - * UsageLogAggregateDisplayInformation.project_name
 - * UsageLogAggregateDisplayInformation.request_count
 - * UsageLogAggregateDisplayInformation.timestamp
 - * UsageLogAggregateDisplayInformation.total_tokens
 - * UsageLogAggregateDisplayInformation.user_name
- UsageLogDisplayInformation
 - * UsageLogDisplayInformation.completion_tokens
 - * UsageLogDisplayInformation.estimated_tokens
 - * UsageLogDisplayInformation.fingerprint
 - * UsageLogDisplayInformation.finish_reason
 - * UsageLogDisplayInformation.gateway_overhead
 - * UsageLogDisplayInformation.id
 - * UsageLogDisplayInformation.internal_cost_estimate
 - * UsageLogDisplayInformation.internal_cost_final
 - * UsageLogDisplayInformation.is_streaming
 - * UsageLogDisplayInformation.model
 - * UsageLogDisplayInformation.project_name
 - * UsageLogDisplayInformation.prompt_tokens
 - * UsageLogDisplayInformation.request_id
 - * UsageLogDisplayInformation.request_type
 - * UsageLogDisplayInformation.status_code
 - * UsageLogDisplayInformation.temperature
 - * UsageLogDisplayInformation.timestamp
 - * UsageLogDisplayInformation.top_k
 - * UsageLogDisplayInformation.top_p
 - * UsageLogDisplayInformation.total_latency
 - * UsageLogDisplayInformation.total_tokens
 - * UsageLogDisplayInformation.ttft
 - * UsageLogDisplayInformation.upstream_latency
 - * UsageLogDisplayInformation.user_name
- UsageLogInformationRequest
 - * UsageLogInformationRequest.aggregate
 - * UsageLogInformationRequest.limit
 - * UsageLogInformationRequest.offset

- * UsageLogInformationRequest.project_id
 - * UsageLogInformationRequest.user_id
- UserDeleteRequest
 - * UserDeleteRequest.user_id
- UserDisplayInformation
 - * UserDisplayInformation.email
 - * UserDisplayInformation.id
 - * UserDisplayInformation.is_admin
 - * UserDisplayInformation.is_password_expired
 - * UserDisplayInformation.is_project_manager
 - * UserDisplayInformation.joined_date
 - * UserDisplayInformation.last_login
 - * UserDisplayInformation.password_expires_at
 - * UserDisplayInformation.profile_picture_url
 - * UserDisplayInformation.role
 - * UserDisplayInformation.username
- UserIdentityChangeRequest
 - * UserIdentityChangeRequest.email
 - * UserIdentityChangeRequest.username
- UserInformationRequest
 - * UserInformationRequest.project_id
- UserPasswordChangeRequest
 - * UserPasswordChangeRequest.current_password
 - * UserPasswordChangeRequest.mandate_reset
 - * UserPasswordChangeRequest.new_password
 - * UserPasswordChangeRequest.new_password_confirm
 - * UserPasswordChangeRequest.user_id
- UserRegistrationRequest
 - * UserRegistrationRequest.email
 - * UserRegistrationRequest.mandate_reset
 - * UserRegistrationRequest.password
 - * UserRegistrationRequest.username
- Submodules
- backend.models.dto_models.key module
 - * ApiKeyDeleteRequest
 - ApiKeyDeleteRequest.key_id
 - * ApiKeyDisplayInformation
 - ApiKeyDisplayInformation.api_key
 - ApiKeyDisplayInformation.create_date
 - ApiKeyDisplayInformation.fingerprint
 - ApiKeyDisplayInformation.id
 - ApiKeyDisplayInformation.name

- ApiKeyDisplayInformation.project_id
 - ApiKeyDisplayInformation.status
 - ApiKeyDisplayInformation.user_id
 - ApiKeyDisplayInformation.username
- * CreateApiKeyRequest
 - CreateApiKeyRequest.name
 - CreateApiKeyRequest.project_id
- * KeyInformationRequest
 - KeyInformationRequest.project_id
 - KeyInformationRequest.user_id
- backend.models.dto_models.login module
 - * AccessTokenInfo
 - AccessTokenInfo.sub
 - AccessTokenInfo.user_id
 - * LoginRequest
 - LoginRequest.email
 - LoginRequest.password
 - * TokenResponse
 - TokenResponse.access_token
 - TokenResponse.token_type
- backend.models.dto_models.logs module
 - * UsageLogAggregateDisplayInformation
 - UsageLogAggregateDisplayInformation.fingerprint
 - UsageLogAggregateDisplayInformation.internal_cost_final
 - UsageLogAggregateDisplayInformation.project_name
 - UsageLogAggregateDisplayInformation.request_count
 - UsageLogAggregateDisplayInformation.timestamp
 - UsageLogAggregateDisplayInformation.total_tokens
 - UsageLogAggregateDisplayInformation.user_name
 - * UsageLogDisplayInformation
 - UsageLogDisplayInformation.completion_tokens
 - UsageLogDisplayInformation.estimated_tokens
 - UsageLogDisplayInformation.fingerprint
 - UsageLogDisplayInformation.finish_reason
 - UsageLogDisplayInformation.gateway_overhead
 - UsageLogDisplayInformation.id
 - UsageLogDisplayInformation.internal_cost_estimate
 - UsageLogDisplayInformation.internal_cost_final
 - UsageLogDisplayInformation.is_streaming
 - UsageLogDisplayInformation.model
 - UsageLogDisplayInformation.project_name
 - UsageLogDisplayInformation.prompt_tokens

- UsageLogDisplayInformation.request_id
- UsageLogDisplayInformation.request_type
- UsageLogDisplayInformation.status_code
- UsageLogDisplayInformation.temperature
- UsageLogDisplayInformation.timestamp
- UsageLogDisplayInformation.top_k
- UsageLogDisplayInformation.top_p
- UsageLogDisplayInformation.total_latency
- UsageLogDisplayInformation.total_tokens
- UsageLogDisplayInformation.ttft
- UsageLogDisplayInformation.upstream_latency
- UsageLogDisplayInformation.user_name
- * UsageLogInformationRequest
 - UsageLogInformationRequest.aggregate
 - UsageLogInformationRequest.limit
 - UsageLogInformationRequest.offset
 - UsageLogInformationRequest.project_id
 - UsageLogInformationRequest.user_id
- backend.models.dto_models.project module
 - * AddUserToProjectRequest
 - AddUserToProjectRequest.project_id
 - AddUserToProjectRequest.user_id
 - * CreateProjectRequest
 - CreateProjectRequest.manager_id
 - CreateProjectRequest.name
 - * ProjectDeleteRequest
 - ProjectDeleteRequest.project_id
 - * ProjectDisplayInfo
 - ProjectDisplayInfo.created_date
 - ProjectDisplayInfo.id
 - ProjectDisplayInfo.modified_date
 - ProjectDisplayInfo.name
 - ProjectDisplayInfo.status
- backend.models.dto_models.quotas module
 - * LimitTypeDisplayInformation
 - LimitTypeDisplayInformation.description
 - LimitTypeDisplayInformation.id
 - LimitTypeDisplayInformation.name
 - * PeriodDisplayInformation
 - PeriodDisplayInformation.name
 - * QuotaCreateRequest
 - QuotaCreateRequest.expires_at

- QuotaCreateRequest.key_id
- QuotaCreateRequest.limit_id
- QuotaCreateRequest.limit_value
- QuotaCreateRequest.period
- QuotaCreateRequest.project_id
- QuotaCreateRequest.user_id
- QuotaCreateRequest.validate_request()
- * QuotaDeleteRequest
 - QuotaDeleteRequest.id
- * QuotaDisplayInformation
 - QuotaDisplayInformation.allocated
 - QuotaDisplayInformation.expires_at
 - QuotaDisplayInformation.fingerprint
 - QuotaDisplayInformation.id
 - QuotaDisplayInformation.limit_id
 - QuotaDisplayInformation.limit_name
 - QuotaDisplayInformation.limit_value
 - QuotaDisplayInformation.name
 - QuotaDisplayInformation.next_reset
 - QuotaDisplayInformation.period
 - QuotaDisplayInformation.project_id
 - QuotaDisplayInformation.status
 - QuotaDisplayInformation.user_name
- * QuotaInformationRequest
 - QuotaInformationRequest.active_only
 - QuotaInformationRequest.include_inherited
 - QuotaInformationRequest.key_id
 - QuotaInformationRequest.project_id
 - QuotaInformationRequest.user_id
 - QuotaInformationRequest.validate_request()

– backend.models.dto_models.user module

- * UserDeleteRequest
 - UserDeleteRequest.user_id
- * UserDisplayInformation
 - UserDisplayInformation.email
 - UserDisplayInformation.id
 - UserDisplayInformation.is_admin
 - UserDisplayInformation.is_password_expired
 - UserDisplayInformation.is_project_manager
 - UserDisplayInformation.joined_date
 - UserDisplayInformation.last_login
 - UserDisplayInformation.password_expires_at
 - UserDisplayInformation.profile_picture_url

- `UserDisplayInformation.role`
- `UserDisplayInformation.username`
- * `UserIdentityChangeRequest`
 - `UserIdentityChangeRequest.email`
 - `UserIdentityChangeRequest.username`
- * `UserInformationRequest`
 - `UserInformationRequest.project_id`
- * `UserPasswordChangeRequest`
 - `UserPasswordChangeRequest.current_password`
 - `UserPasswordChangeRequest.mandate_reset`
 - `UserPasswordChangeRequest.new_password`
 - `UserPasswordChangeRequest.new_password_confirm`
 - `UserPasswordChangeRequest.user_id`
- * `UserRegistrationRequest`
 - `UserRegistrationRequest.email`
 - `UserRegistrationRequest.mandate_reset`
 - `UserRegistrationRequest.password`
 - `UserRegistrationRequest.username`

13 backend.routes.auth package

13.1 Submodules

13.2 backend.routes.auth.login module

13.2.1 *async* login(request: LoginRequest)

Authenticates a user based on email and password, and returns a JWT access token if the credentials are valid.

- **Parameters:** `request` (*LoginRequest*) – A *LoginRequest* object containing the user’s email and password.
- **Returns:** An object containing the access token and token type if authentication is successful.
- **Return type:** *TokenResponse*

13.3 backend.routes.auth.register module

13.3.1 *async* register(request: UserRegistrationRequest, _: UserDisplayInformation = fastapi.Depends)

Registers a new user with the provided information. This endpoint is protected and requires the requester to be an authenticated administrator user.

- **Parameters:**

- **request** (*UserRegistrationRequest*) – A *UserRegistrationRequest* object containing the new user’s email, username, password, and `mandate_reset` flag.
- **_** (*UserDisplayInformation*) – An unused *UserDisplayInformation* object injected by the `require_administrator_user` dependency to enforce admin permissions.
- **Returns:** A dictionary containing a success message with the new user’s username if registration is successful.
- **Return type:** *dict*

14 backend.routes.keys package

14.1 Submodules

14.2 backend.routes.keys.create module

14.2.1 *async* `create_api_key(request: CreateApiKeyRequest, current_user: UserDisplayInformation = fastapi.Depends)`

Create a new API key for a project accessible by the current user.

- **Parameters:**
 - **request** (*CreateApiKeyRequest*) – API key creation payload including project identifier and key name.
 - **current_user** (*UserDisplayInformation*) – Authenticated user resolved from access token.
- **Returns:** *ApiKeyDisplayInformation* containing metadata and the generated raw API key value.
- **Return type:** *ApiKeyDisplayInformation*

14.3 backend.routes.keys.delete module

14.3.1 *async* `delete_api_key(request: ApiKeyDeleteRequest, current_user: UserDisplayInformation = fastapi.Depends)`

Archive an API key after checking the current user’s deletion permission.

- **Parameters:**
 - **request** (*ApiKeyDeleteRequest*) – API key deletion payload containing the target key identifier.
 - **current_user** (*UserDisplayInformation*) – Authenticated user attempting the deletion.
- **Returns:** A success message dictionary when archival completes.
- **Return type:** *dict*

14.4 backend.routes.keys.info module

14.4.1 *async* `get_key_information(request: KeyInformationRequest = fastapi.Depends, _: None = fastapi.Depends) → list[ApiKeyDisplayInformation]`

Retrieve API keys scoped either to a project or to a specific user or a specified user's keys in a project.

- **Parameters:**

- **request** (*KeyInformationRequest*) – Query parameters selecting project or user scope.
- **_** (*None*) – Token validation dependency output, unused in function body.

- **Returns:** A list of API key display models matching the requested scope.

- **Return type:** *list[ApiKeyDisplayInformation]*

15 backend.routes package

15.1 Subpackages

- backend.routes.auth package
 - Submodules
 - backend.routes.auth.login module
 - * `login()`
 - backend.routes.auth.register module
 - * `register()`
- backend.routes.keys package
 - Submodules
 - backend.routes.keys.create module
 - * `create_api_key()`
 - backend.routes.keys.delete module
 - * `delete_api_key()`
 - backend.routes.keys.info module
 - * `get_key_information()`
- backend.routes.projects package
 - Submodules
 - backend.routes.projects.add_user module
 - * `add_user_to_project()`
 - backend.routes.projects.all module
 - * `get_all_projects()`

- backend.routes.projects.create module
 - * `create_project()`
- backend.routes.projects.delete module
 - * `delete_project()`
- backend.routes.projects.info module
 - * `get_project_information()`
- backend.routes.projects.remove_user module
 - * `remove_user_from_project()`
- backend.routes.quotas package
 - Submodules
 - backend.routes.quotas.create module
 - * `create_quota()`
 - backend.routes.quotas.delete module
 - * `delete_quota()`
 - backend.routes.quotas.info module
 - * `get_user_information()`
 - backend.routes.quotas.limit_types module
 - * `get_limit_types()`
 - backend.routes.quotas.periods module
 - * `get_periods()`
- backend.routes.usage_logs package
 - Submodules
 - backend.routes.usage_logs.info module
 - * `get_log_information()`
- backend.routes.users package
 - Submodules
 - backend.routes.users.change_identity module
 - * `change_identity()`
 - backend.routes.users.change_password module
 - * `change_password()`
 - backend.routes.users.delete module
 - * `delete_user_route()`
 - backend.routes.users.everyone module
 - * `get_all_users()`
 - backend.routes.users.info module
 - * `get_user_information()`
 - backend.routes.users.me module
 - * `get_current_user()`

15.2 Submodules

15.3 backend.routes.health module

15.3.1 *async* health(request: fastapi.Request)

Endpoint to check the health of the gateway.

- **Returns:** A simple JSON response indicating OK status and optionally the version of the gateway.
- **Return type:** *dict*

15.3.2 *async* root(request: fastapi.Request)

Alias for the health check endpoint, allowing the gateway status to be checked by sending a request to the root path.

SEE ALSO

health The main health check endpoint that this function calls to get the gateway status.

16 backend.routes.projects package

16.1 Submodules

16.2 backend.routes.projects.add_user module

16.2.1 *async* add_user_to_project(request: AddUserToProjectRequest, credentials: HTTPAuthorizationCredentials | None = fastapi.Depends)

Add a user to a project after verifying manager permissions.

- **Parameters:**
 - **request** (*AddUserToProjectRequest*) – Payload containing project and user identifiers.
 - **credentials** (*HTTPAuthorizationCredentials* | *None* , *optional*) – Optional bearer credentials used for permission validation.
- **Returns:** A success message dictionary when membership is created.
- **Return type:** *dict*

16.3 backend.routes.projects.all module

16.3.1 *async* get_all_projects(_: None = fastapi.Depends) → list[ProjectDisplayInfo]

Retrieve all visible projects in the system.

- **Parameters:** `_` (*None*) – Token validation dependency output, unused in function body.
- **Returns:** A list of project display models.
- **Return type:** *list[ProjectDisplayInfo]*

16.4 backend.routes.projects.create module

16.4.1 *async* create_project(request: CreateProjectRequest, _: UserDisplayInformation = fastapi.Depends)

Create a new project and assign its initial manager.

- **Parameters:**
 - **request** (*CreateProjectRequest*) – Project creation payload including name and manager user identifier.
 - `_` (*UserDisplayInformation*) – Administrator authorization dependency output, unused in function body.
- **Returns:** A success message dictionary containing the created project name.
- **Return type:** *dict*

16.5 backend.routes.projects.delete module

16.5.1 *async* delete_project(request: ProjectDeleteRequest, credentials: HTTPAuthorizationCredentials | None = fastapi.Depends)

Archive an existing project after verifying manager permissions.

- **Parameters:**
 - **request** (*ProjectDeleteRequest*) – Payload containing the project identifier to archive.
 - **credentials** (*HTTPAuthorizationCredentials | None*, *optional*) – Optional bearer credentials used for permission validation.
- **Returns:** A success message dictionary when archival completes.
- **Return type:** *dict*

16.6 backend.routes.projects.info module

16.6.1 *async* get_project_information(current_user: UserDisplayInformation = fastapi.Depends) → list[ProjectDisplayInfo]

Retrieve project information scoped to the current user role.

- **Parameters:** **current_user** (*UserDisplayInformation*) – Authenticated user used to determine visibility scope.

- **Returns:** A list of project display models for all projects (admin) or memberships (non-admin).
- **Return type:** *list[ProjectDisplayInfo]*

16.7 backend.routes.projects.remove_user module

16.7.1 *async* `remove_user_from_project(request: AddUserToProjectRequest, credentials: HTTPAuthorizationCredentials | None = fastapi.Depends)`

Remove a user from a project after verifying manager permissions.

- **Parameters:**
 - **request** (*AddUserToProjectRequest*) – Payload containing project and user identifiers.
 - **credentials** (*HTTPAuthorizationCredentials* | *None*, optional) – Optional bearer credentials used for permission validation.
- **Returns:** A success message dictionary when membership removal completes.
- **Return type:** *dict*

17 backend.routes.quotas package

17.1 Submodules

17.2 backend.routes.quotas.create module

17.2.1 *async* `create_quota(request: QuotaCreateRequest, current_user: UserDisplayInformation = fastapi.Depends) → QuotaDisplayInformation`

Create a quota for a valid project, user, or API key target.

- **Parameters:**
 - **request** (*QuotaCreateRequest*) – Quota creation payload with target, limit, period, and optional expiry.
 - **current_user** (*UserDisplayInformation*) – Authenticated user used for permission enforcement.
- **Returns:** *QuotaDisplayInformation* for the newly created quota.
- **Return type:** *QuotaDisplayInformation*

17.3 backend.routes.quotas.delete module

17.3.1 *async* `delete_quota(request: QuotaDeleteRequest, current_user: UserDisplayInformation = fastapi.Depends)`

Delete a quota after validating the current user's permission.

- **Parameters:**
 - **request** (*QuotaDeleteRequest*) – Quota deletion payload containing the quota identifier.
 - **current_user** (*UserDisplayInformation*) – Authenticated user requesting the deletion.
- **Returns:** A success message dictionary when deletion completes.
- **Return type:** *dict*

17.4 backend.routes.quotas.info module

17.4.1 *async* **get_user_information**(request: *QuotaInformationRequest* = fastapi.Depends, _: None = fastapi.Depends) → list[*QuotaDisplayInformation*]

Retrieve quotas globally or filtered by project, user, or API key.

- **Parameters:**
 - **request** (*QuotaInformationRequest*) – Query payload selecting scope and inclusion flags.
 - **_** (*None*) – Token validation dependency output, unused in function body.
- **Returns:** A list of quota display models matching the requested scope.
- **Return type:** list[*QuotaDisplayInformation*]

17.5 backend.routes.quotas.limit_types module

17.5.1 *async* **get_limit_types**(_: None = fastapi.Depends) → list[*LimitTypeDisplayInformation*]

Retrieve all configured quota limit types.

- **Parameters:** **_** (*None*) – Token validation dependency output, unused in function body.
- **Returns:** A list of limit type display models.
- **Return type:** list[*LimitTypeDisplayInformation*]

17.6 backend.routes.quotas.periods module

17.6.1 *async* **get_periods**(_: None = fastapi.Depends) → list[*PeriodDisplayInformation*]

Retrieve all supported quota period enum values.

- **Parameters:** **_** (*None*) – Token validation dependency output, unused in function body.
- **Returns:** A list of period display models.
- **Return type:** list[*PeriodDisplayInformation*]

18 backend.routes.usage_logs package

18.1 Submodules

18.2 backend.routes.usage_logs.info module

18.2.1 *async* `get_log_information(request: UsageLogInformationRequest = fastapi.Depends, _: None = fastapi.Depends) → list[UsageLogDisplayInformation] | list[UsageLogAggregateDisplayInformation]`

Retrieve usage logs by scope, with optional aggregation mode.

- **Parameters:**

- **request** (*UsageLogInformationRequest*) – Query payload with filters, pagination, and aggregation options.
- **_** (*None*) – Token validation dependency output, unused in function body.

- **Returns:** A list of detailed usage logs or aggregated rows based on request.aggregate.

- **Return type:** `list[UsageLogDisplayInformation] | list[UsageLogAggregateDisplayInformation]`

19 backend.routes.users package

19.1 Submodules

19.2 backend.routes.users.change_identity module

19.2.1 *async* `change_identity(request: UserIdentityChangeEventRequest, current_user: UserDisplayInformation = fastapi.Depends)`

Update the authenticated user's email and or username.

- **Parameters:**

- **request** (*UserIdentityChangeEventRequest*) – Identity update payload containing new email and or username values.
- **current_user** (*UserDisplayInformation*) – Authenticated user whose profile is being updated.

- **Returns:** A success message dictionary when profile data is updated.

- **Return type:** *dict*

19.3 backend.routes.users.change_password module

19.3.1 *async* change_password(request: UserPasswordChangeRequest, current_user: UserDisplayInformation = fastapi.Depends)

Change a password for the current user or, for admins, another user.

- **Parameters:**

- **request** (*UserPasswordChangeRequest*) – Password change payload with current and new password fields.
- **current_user** (*UserDisplayInformation*) – Authenticated user requesting the password change.

- **Returns:** A success message dictionary when password update completes.

- **Return type:** *dict*

19.4 backend.routes.users.delete module

19.4.1 *async* delete_user_route(request: UserDeleteRequest, _: UserDisplayInformation = fastapi.Depends)

Delete a user account by identifier.

- **Parameters:**

- **request** (*UserDeleteRequest*) – User deletion payload containing the target user identifier.
- **_** (*UserDisplayInformation*) – Administrator authorization dependency output, unused in function body.

- **Returns:** A success message dictionary when deletion completes.

- **Return type:** *dict*

19.5 backend.routes.users.everyone module

19.5.1 *async* get_all_users(_: None = fastapi.Depends) → list[UserDisplayInformation]

Retrieve all users available in the dashboard scope.

- **Parameters:** **_** (*None*) – Token validation dependency output, unused in function body.
- **Returns:** A list of user display models.
- **Return type:** *list[UserDisplayInformation]*

19.6 backend.routes.users.info module

19.6.1 *async* `get_user_information(request: UserInformationRequest = fastapi.Depends, _: UserDisplayInformation = fastapi.Depends) → list[UserDisplayInformation]`

Retrieve user information globally or filtered by project membership.

- **Parameters:**

- **request** (*UserInformationRequest*) – Query payload containing optional project scope.
- **_** (*UserDisplayInformation*) – Token validation dependency output, unused in function body.

- **Returns:** A list of user display models with optional role context.

- **Return type:** *list[UserDisplayInformation]*

19.7 backend.routes.users.me module

19.7.1 *async* `get_current_user(credentials: fastapi.security.HTTPAuthorizationCredentials = fastapi.Depends, project_id: int | None = None) → UserDisplayInformation`

Return the current authenticated user's profile information.

- **Parameters:**

- **credentials** (*HTTPAuthorizationCredentials*) – Bearer authorization credentials containing the access token.
- **project_id** (*Optional [**int* *], optional*) – Optional project context used when resolving role-specific fields.

- **Returns:** *UserDisplayInformation* for the token subject.

- **Return type:** *UserDisplayInformation*

20 backend

- backend package

- Subpackages

- * backend.app package

- `create_app()`
 - Submodules
 - backend.app.factory module
 - backend.app.middleware module

- * backend.auth package

- `create_access_token()`

- get_user_from_token()
- require_administrator_user()
- require_current_user()
- require_project_manager_user()
- require_valid_access_token()
- verify_access_token()
- Submodules
- backend.auth.access_token module

* backend.db package

- add_user_to_project()
- change_user_identity()
- change_user_password()
- create_key()
- create_project()
- create_quota()
- create_user()
- delete_key()
- delete_project()
- delete_quota()
- delete_user()
- get_all_keys()
- get_all_limits()
- get_all_projects()
- get_all_users()
- get_global_quotas()
- get_key_by_id()
- get_key_ownership()
- get_keys_for_project()
- get_keys_for_user()
- get_limit_by_id()
- get_project_by_id()
- get_project_by_name()
- get_projects_for_user()
- get_quota_by_id()
- get_quotas_for_api_key()
- get_quotas_for_project()
- get_quotas_for_user()
- get_role_by_name()
- get_usage_logs()
- get_user_by_email()
- get_user_by_id()
- get_user_by_username()
- get_user_permissions_for_project()
- get_users_by_project()
- is_password_expired()

- is_user_administrator()
- is_user_project_manager()
- is_user_project_member()
- remove_user_from_project()
- user_email_free()
- user_username_free()
- Submodules
 - backend.db.keys module
 - backend.db.logs module
 - backend.db.lookups module
 - backend.db.permissions module
 - backend.db.projects module
 - backend.db.quotas module
 - backend.db.roles module
 - backend.db.users module
- * backend.management package
 - enforce_password_change_validity()
 - enforce_password_strength()
 - generate_api_key()
 - key_convert_orm_to_display_info()
 - key_enforce_deletion_permission()
 - log_convert_aggregate_row_to_display_info()
 - log_convert_orm_to_display_info()
 - project_convert_orm_to_display_info()
 - project_enforce_availability()
 - project_enforce_existing_project()
 - project_enforce_user_not_in_project()
 - quota_convert_limit_orm_to_display_info()
 - quota_convert_orm_to_display_info()
 - quota_convert_period_enum_to_display_info()
 - quota_enforce_creation_permission()
 - quota_enforce_deletion_permission()
 - quota_enforce_existing_limit()
 - quota_enforce_existing_quota_target()
 - user_convert_orm_to_display_info()
 - user_enforce_availability()
 - user_enforce_existing_user()
 - Subpackages
- * backend.models package
 - AccessTokenInfo
 - AddUserToProjectRequest
 - ApiKeyDeleteRequest
 - ApiKeyDisplayInformation
 - CreateApiKeyRequest
 - CreateProjectRequest

- KeyInformationRequest
- LimitTypeDisplayInformation
- LoginRequest
- PeriodDisplayInformation
- ProjectDeleteRequest
- ProjectDisplayInfo
- QuotaCreateRequest
- QuotaDeleteRequest
- QuotaDisplayInformation
- QuotaInformationRequest
- TokenResponse
- UsageLogAggregateDisplayInformation
- UsageLogDisplayInformation
- UsageLogInformationRequest
- UserDeleteRequest
- UserDisplayInformation
- UserIdentityChangeRequest
- UserInformationRequest
- UserPasswordChangeRequest
- UserRegistrationRequest
- Subpackages
- * backend.routes package
 - Subpackages
 - Submodules
 - backend.routes.health module